



Faculty of Engineering
Mobile Application Development
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: **BRANCH:**

SEMESTER: **ACADEMIC YEAR:**



Swaminarayan university
Publication

ISBN Number: 978-81-998196-2-7



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in has satisfactorily completed his
term working (Subject name) (subject code:)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ISHANI HARSHIL DAVE
SUBJECT IN-CHARGE

NIMESH VAIDYA
HOD

DR. VIJAYKUMAR. B. GADHAVI
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1	Set-up of Android development environment, managing AVD and understanding its various components.			
2	Understanding of Various Components available in Android Application			
3	Develop a “Hello World” Application in Android and understand the structure of an Android Application			
4	Develop Android Application to demonstrate methods of Activity Life Cycle			
5	Design Android Activities using LinearLayout, RelativeLayout, GridView, FrameLayout, and ConstraintLayout			
6	Design various Activities using different Layouts and available Widgets(Text View, EditText, Button, RadioButton, CheckBox, ImageButton, ToggleButton, TimePicker, DatePicker, ProgressBar, ImageView) to make the user-friendly GUI.			
7	Develop code to demonstrate different ways of Handling different events (onClick, onLongClick etc.) over Button, EditText etc. to perform action in Android application at run-time.			
8	Develop code to demonstrate Event handling of CheckBox and RadioButton selection.			
9	Develop code to navigate between different activities and pass the data from one activity to other activity using Intent.			
10	Develop an android application to store data locally using SharedPreferences and access-modify in different activities.			
11	Develop the code to implement the ListView and the Spinner views, perform add, update, remove items operations and implement the item selection event handling over ListView and Spinner for appropriate example.			
12	Develop the code to manage Permission using Manifest file and run time from Activity, and toggle state of WiFi and Bluetooth.			

13	Develop android applications to demonstrate user interaction with the application using Options Menu, Context Menu and Popup Menu.			
14	Develop Android Applications to demonstrate different AlertDialogs and the Custom Dialog.			
15	Develop Android Application for local database connectivity and performing basic database operations (select, insert, update, delete) using SQLiteDatabase and SQLiteOpenHelper Classes			
16	Develop an Android Application to demonstrate the use of RecyclerView and CardView for displaying list of items with multiple information			
17	Publish an Android Application on Play Store			

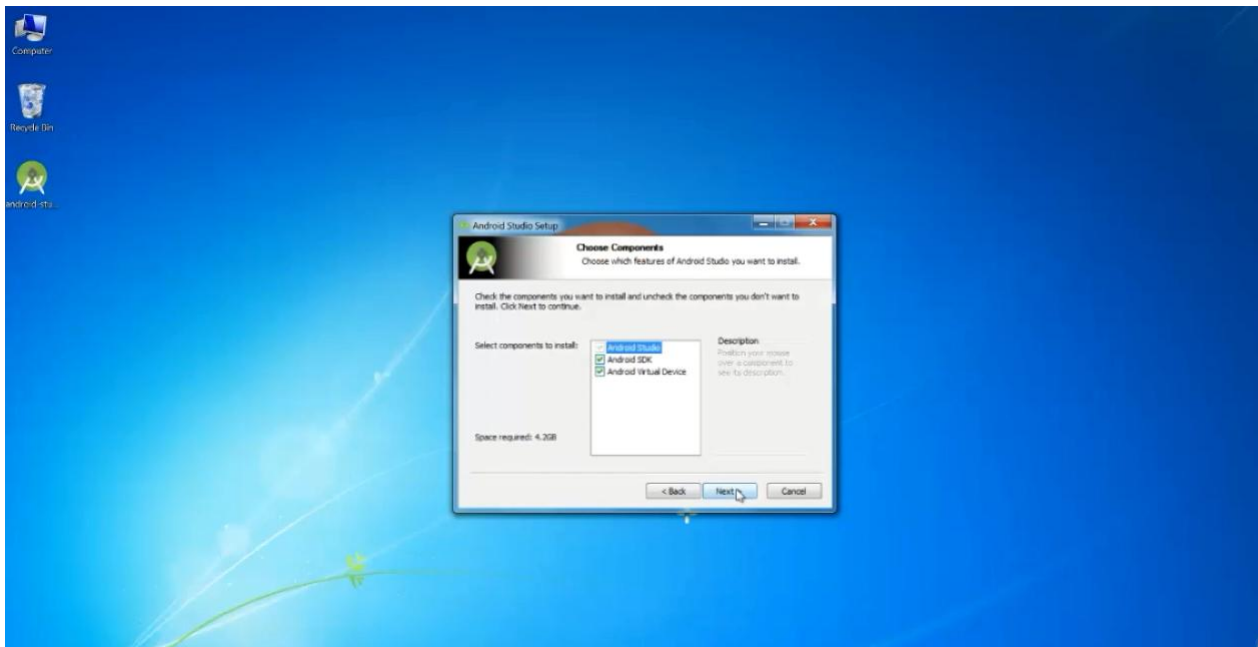
PRACTICAL-1

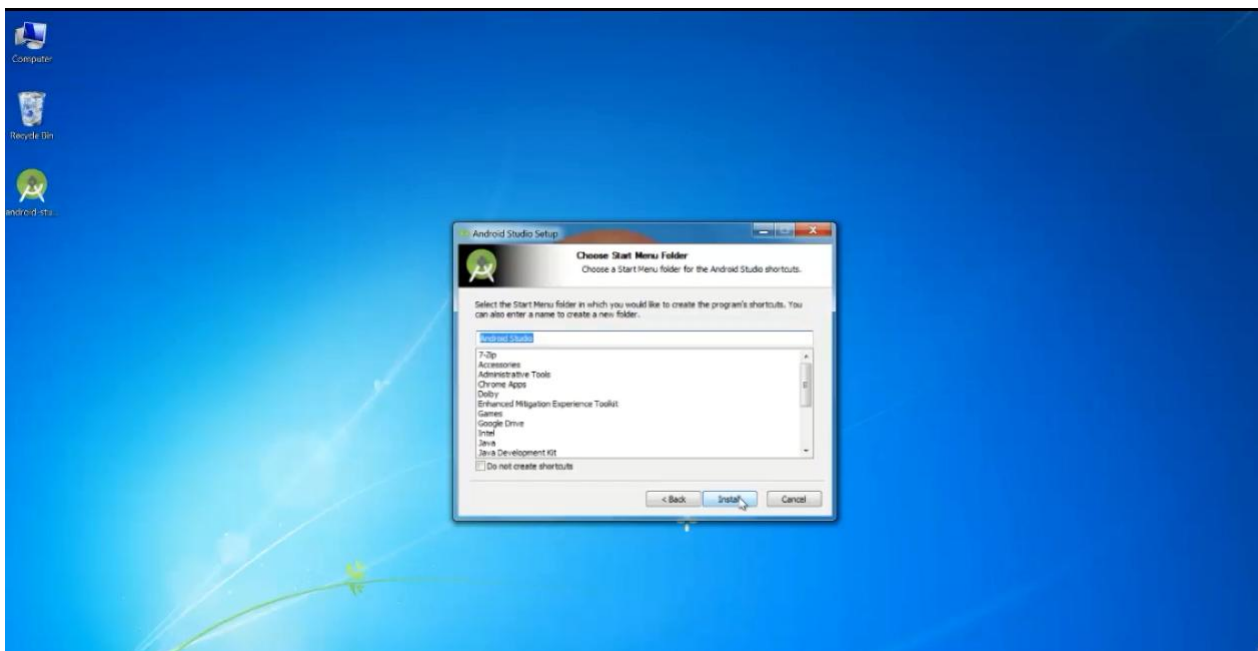
AIM:- Set-up of Android development environment, managing AVD and understanding its various components.

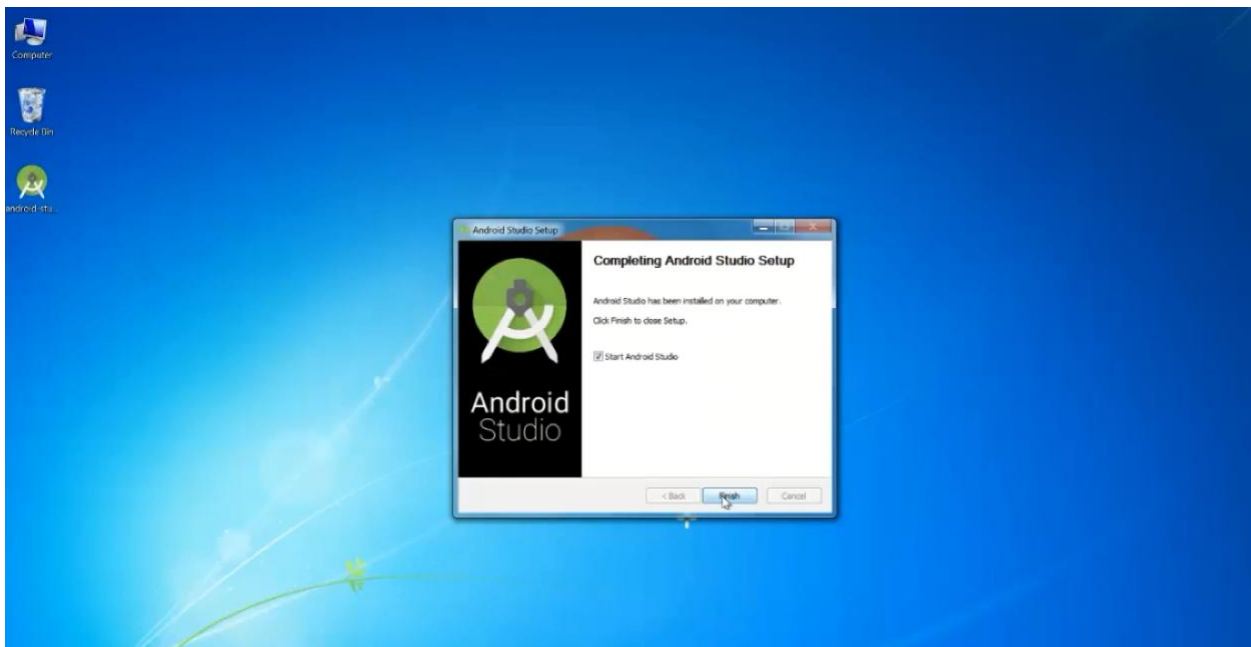
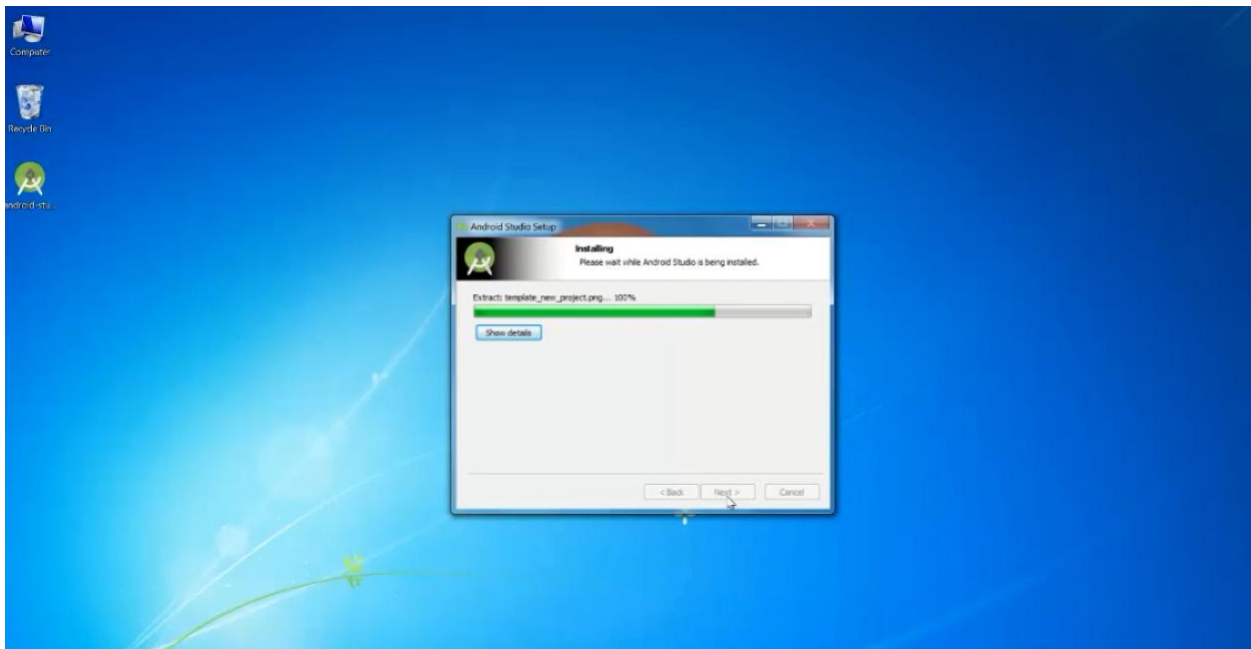
- **Set up your development environment**

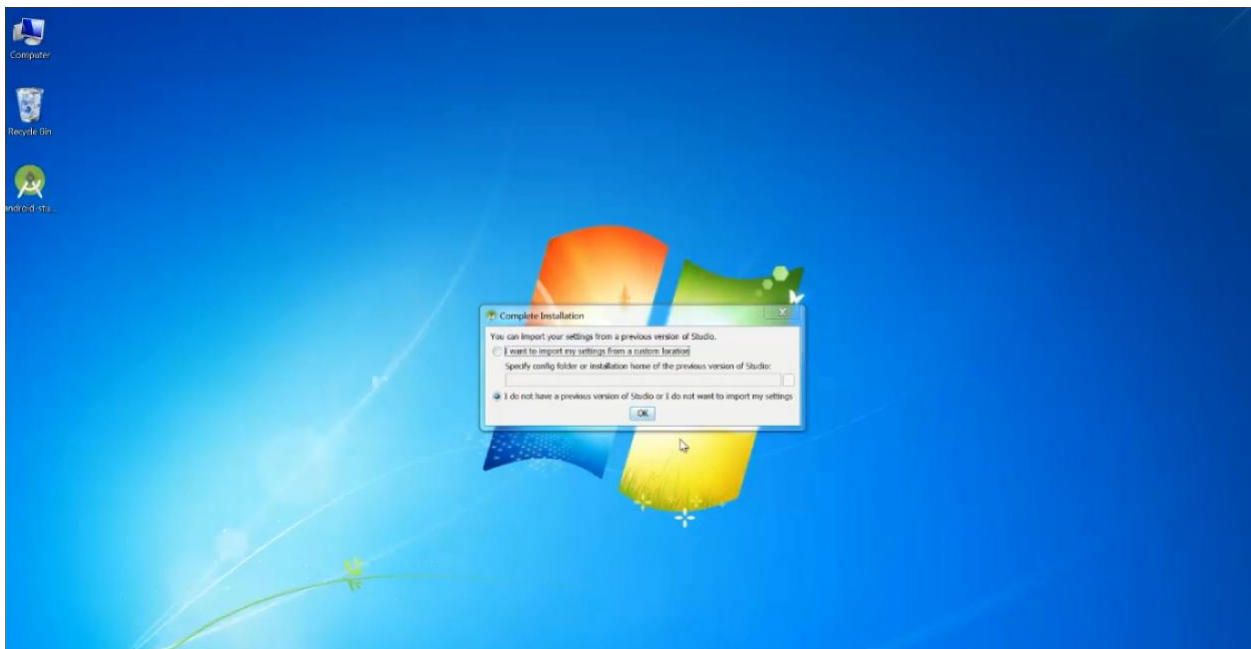
1. Install the latest Canary build or Stable build of Android Studio.
2. In Android Studio, go to Tools > SDK Manager.
3. In the SDK Platforms tab, select Android UpsideDownCakePrivacySandbox Preview.
4. In the SDK Tools tab, select Android SDK Build-Tools 33 or later.
5. Click **OK** to install the SDK.

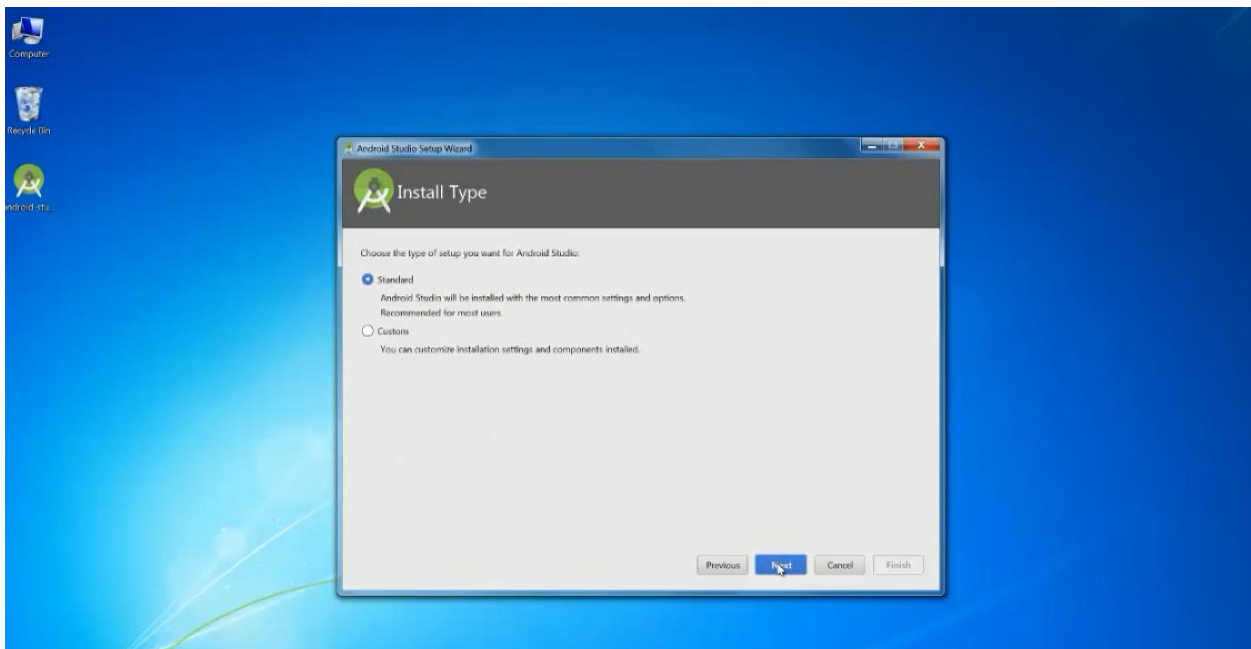
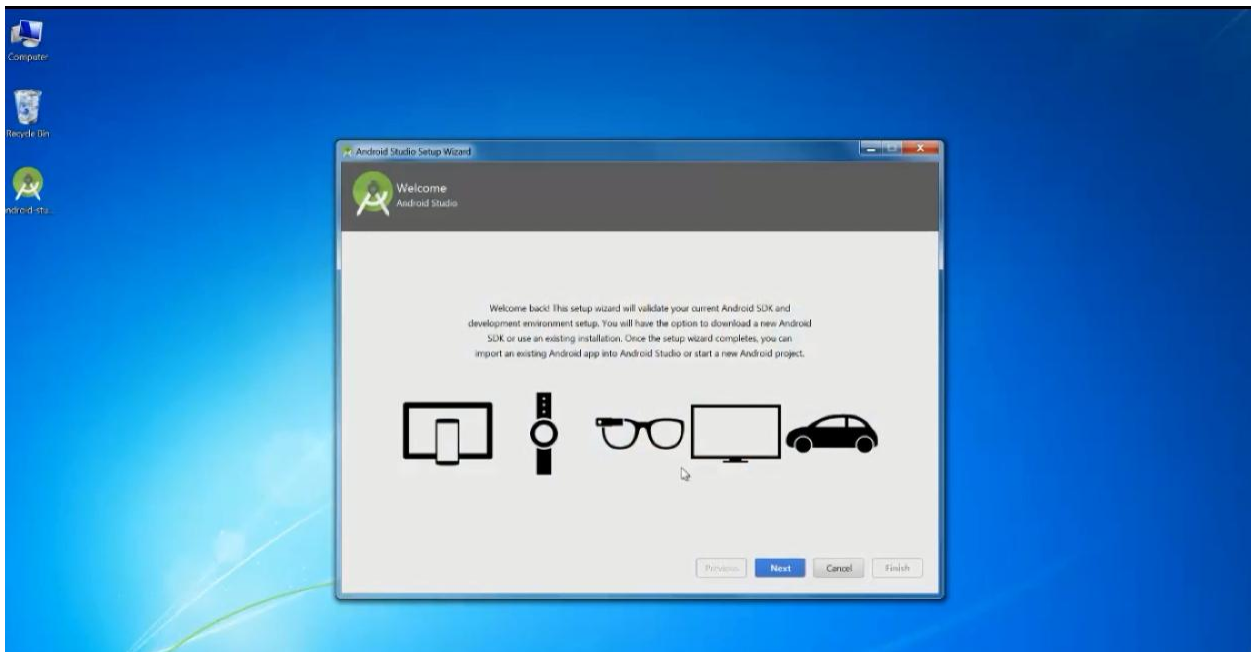


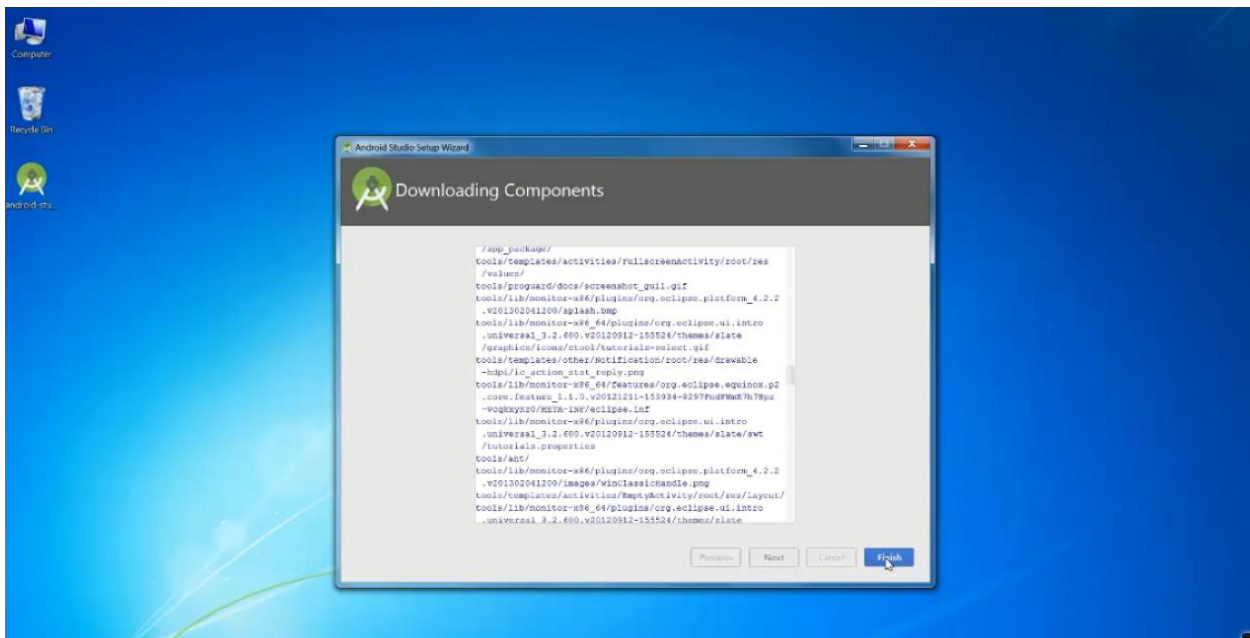
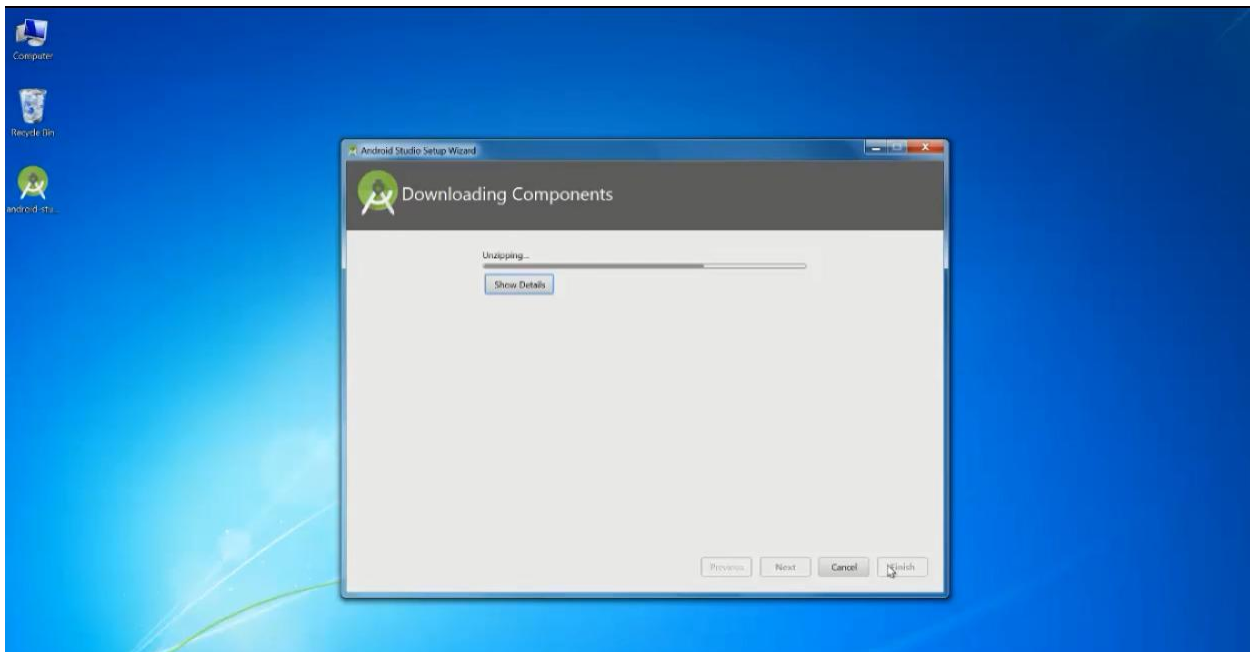


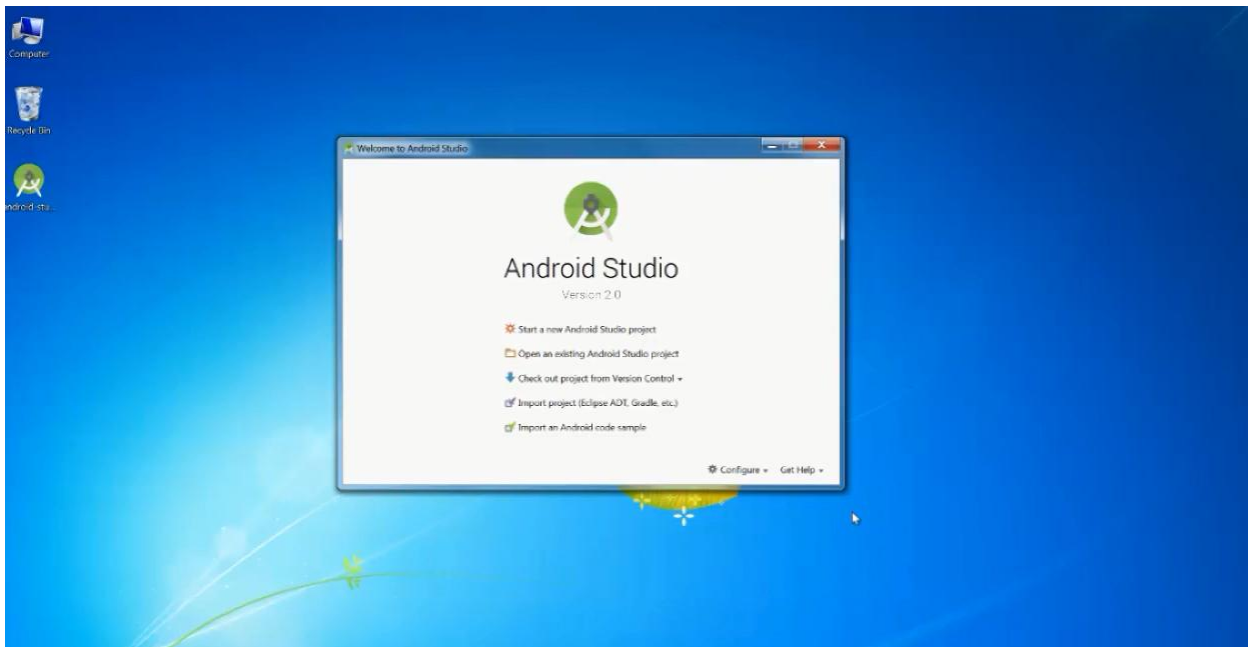












- **Azure Virtual Desktop key management tasks**

Once you've set up AVD, we've identified a number of key management tasks to keep the environment performing at its best, including how to:

- Administer Group Policy
- Manage images
- Update your desktops and applications or deploy new applications
- Validate updates before users access them
- Add or remove session hosts from the host pool
- Create new host pools
- Monitor and receive alerts for the environment
- Backup the environment

PRACTICAL -2

AIM :- Understanding of Various Components available in Android Application

Application components are the essential building blocks of an Android application. These components are loosely coupled by the application manifest file *AndroidManifest.xml* that describes each component of the application and how they interact.

There are following four main components that can be used within an Android application –

Sr.No	Components & Description
1	Activities They dictate the UI and handle the user interaction to the smart phone screen.
2	Services They handle background processing associated with an application.
3	Broadcast Receivers They handle communication between Android OS and applications.
4	Content Providers They handle data and database management issues.

Activities

An activity represents a single screen with a user interface, in short Activity performs actions on the screen. For example, an email application might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails. If an application has more than one activity, then one of them should be marked as the activity that is presented when the application is launched.

An activity is implemented as a subclass of **Activity** class as follows –

```
public class MainActivity extends Activity {  
}
```

Services

A service is a component that runs in the background to perform long-running operations. For example, a service might play music in the background while the user is in a different application, or it might fetch data over the network without blocking user interaction with an activity.

A service is implemented as a subclass of **Service** class as follows –

```
public class MyService extends Service {
}
```

Broadcast Receivers

Broadcast Receivers simply respond to broadcast messages from other applications or from the system. For example, applications can also initiate broadcasts to let other applications know that some data has been downloaded to the device and is available for them to use, so this is broadcast receiver who will intercept this communication and will initiate appropriate action.

A broadcast receiver is implemented as a subclass of **BroadcastReceiver** class and each message is broadcaster as an **Intent** object.

```
public class MyReceiver extends BroadcastReceiver {
    public void onReceive(context,intent){}
}
```

Content Providers

A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the *ContentResolver* class. The data may be stored in the file system, the database or somewhere else entirely.

A content provider is implemented as a subclass of **ContentProvider** class and must implement a standard set of APIs that enable other applications to perform transactions.

```
public class MyContentProvider extends ContentProvider {
    public void onCreate(){}
}
```

We will go through these tags in detail while covering application components in individual chapters.

Additional Components

There are additional components which will be used in the construction of above mentioned entities, their logic, and wiring between them. These components are –

S.No	Components & Description
1	Fragments Represents a portion of user interface in an Activity.

2	Views UI elements that are drawn on-screen including buttons, lists forms etc.
3	Layouts View hierarchies that control screen format and appearance of the views.
4	Intents Messages wiring components together.
5	Resources External elements, such as strings, constants and drawable pictures.
6	Manifest Configuration file for the application.

PRACTICAL-3

AIM :- Develop a “Hello World” Application in Android and understand the structure of an Android Application

The Main Activity File

The main activity code is a Java file **MainActivity.java**. This is the actual application file which ultimately gets converted to a Dalvik executable and runs your application. Following is the default code generated by the application wizard for *Hello World!* application –

```
package com.example.helloworld;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.tutorialspoint7.myapplication">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

- <activity>elements for activities
- <service> elements for services
- <receiver> elements for broadcast receivers
- <provider> elements for content providers

The Strings File

The **strings.xml** file is located in the *res/values* folder and it contains all the text that your application uses. For example, the names of buttons, labels, default text, and similar types of strings go into this file. This file is responsible for their textual content. For example, a default strings file will look like as following file –

```
<resources>
  <string name="app_name">HelloWorld</string>
  <string name="hello_world">Hello world!</string>
  <string name="menu_settings">Settings</string>
  <string name="title_activity_main">MainActivity</string>
</resources>
```

The Layout File

The **activity_main.xml** is a layout file available in *res/layout* directory, that is referenced by your application when building its interface. You will modify this file very frequently to change the layout of your application. For your "Hello World!" application, this file will have following content related to default layout –

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent" >

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_centerVertical="true"
    android:padding="@dimen/padding_medium"
    android:text="@string/hello_world"
    tools:context=".MainActivity" />

</RelativeLayout>
```

OUTPUT:-



PRACTICAL-4

AIM:- Develop Android Application to demonstrate methods of Activity Life Cycle

Demo Android App to Demonstrate Activity Lifecycle in Android

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.widget.Toast;
```

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        Toast toast = Toast.makeText(getApplicationContext(), "onCreate Called",  
        Toast.LENGTH_LONG).show();
```

```
    }
```

```
    protected void onStart() {
```

```
        super.onStart();
```

```
        Toast toast = Toast.makeText(getApplicationContext(), "onStart Called",  
        Toast.LENGTH_LONG).show();
```

```
    }
```

```
    @Override
```

```
    protected void onRestart() {
```

```
        super.onRestart();
```

```
        Toast toast = Toast.makeText(getApplicationContext(), "onRestart Called",  
        Toast.LENGTH_LONG).show();
```

```
    }

    protected void onPause() {
        super.onPause();

        Toast toast = Toast.makeText(getApplicationContext(), "onPause Called",
Toast.LENGTH_LONG).show();
    }

    protected void onResume() {
        super.onResume();

        Toast toast = Toast.makeText(getApplicationContext(), "onResume Called",
Toast.LENGTH_LONG).show();
    }

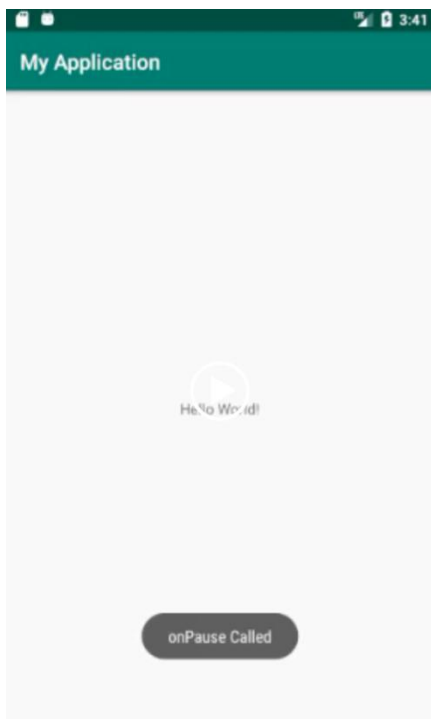
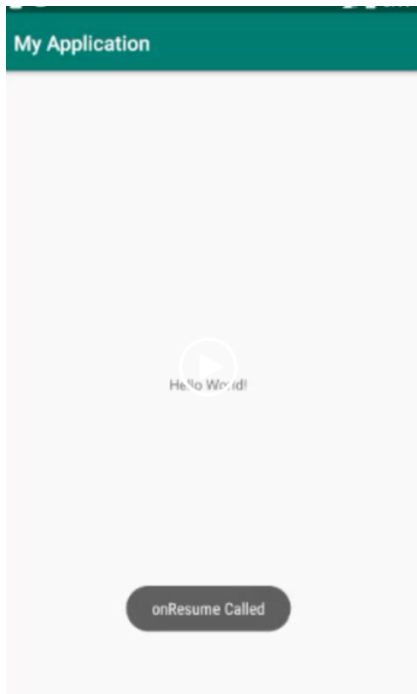
    protected void onStop() {
        super.onStop();

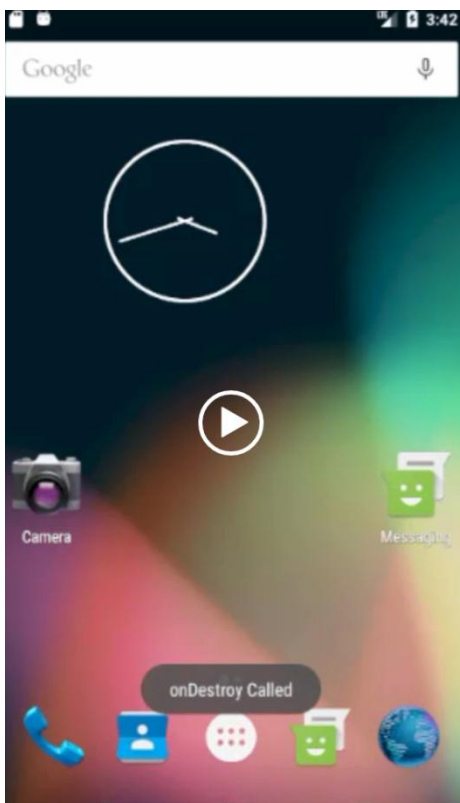
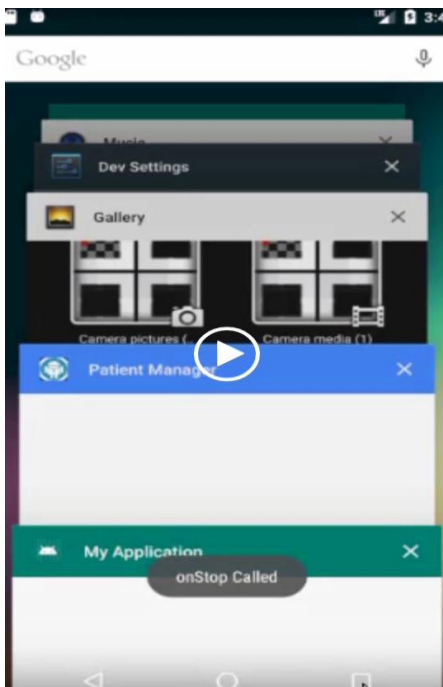
        Toast toast = Toast.makeText(getApplicationContext(), "onStop Called",
Toast.LENGTH_LONG).show();
    }

    protected void onDestroy() {
        super.onDestroy();

        Toast toast = Toast.makeText(getApplicationContext(), "onDestroy Called",
Toast.LENGTH_LONG).show();
    }
}
```

OUTPUT :-





PRACTICAL-5

AIM:- Design Android Activities using LinearLayout, RelativeLayout, GridView, FrameLayout, and ConstraintLayout

Android LinearLayout Declaration

Following is the way we need to define the LinearLayout in android applications.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <!-- Add Child Views Here -->
</LinearLayout>
```

activity_main.xml

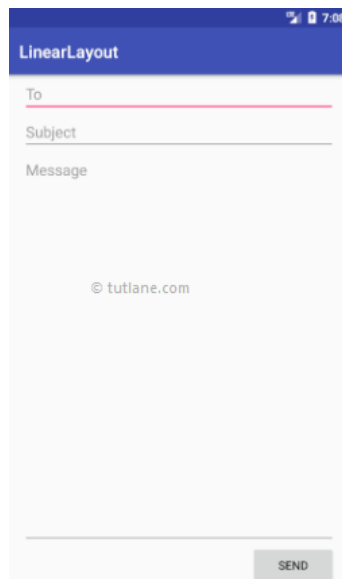
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="20dp"
    android:paddingRight="20dp"
    android:orientation="vertical" >
    <EditText
        android:id="@+id/txtTo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="To"/>
    <EditText
        android:id="@+id/txtSub"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Subject"/>
    <EditText
        android:id="@+id/txtMsg"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="top"
        android:hint="Message"/>
    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
```

```
        android:layout_gravity="right"
        android:text="Send"/>
</LinearLayout>
```

MainActivity.java

```
package com.tutlane.linearlayout;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



Android RelativeLayout Example

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
android:layout_height="match_parent"
android:paddingLeft="10dp"
android:paddingRight="10dp">
<Button
    android:id="@+id/btn1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentLeft="true"
    android:text="Button1" />
<Button
    android:id="@+id/btn2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentRight="true"
    android:layout_centerVertical="true"
    android:text="Button2" />
<Button
    android:id="@+id/btn3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentLeft="true"
    android:layout_centerVertical="true"
    android:text="Button3" />

<Button
    android:id="@+id/btn4"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentBottom="true"
    android:text="Button4" />
<Button
    android:id="@+id/btn5"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignBottom="@+id/btn2"
    android:layout_centerHorizontal="true"
    android:text="Button5" />
<Button
    android:id="@+id/btn6"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_above="@+id/btn4"
    android:layout_centerHorizontal="true"
    android:text="Button6" />
<Button
    android:id="@+id/btn7"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_toEndOf="@+id/btn1"
```

```
        android:layout_toRightOf="@+id/btn1"  
        android:layout_alignParentRight="true"  
        android:text="Button7" />  
</RelativeLayout>
```

MainActivity.java

```
package com.tutlane.linearlayout;  
import android.support.v7.app.AppCompatActivity;  
import android.os.Bundle;  
  
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

Output :-



Android FrameLayout Example

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <ImageView
        android:id="@+id/imgvw1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:scaleType="centerCrop"
        android:src="@drawable/flimg" />
    <TextView
        android:id="@+id/txtvw1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="40dp"
        android:background="#4C374A"
        android:padding="10dp"
        android:text="Grand Palace, Bangkok"
        android:textColor="#FFFFFF"
        android:textSize="20sp" />
    <TextView
        android:id="@+id/txtvw2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="right|bottom"
        android:background="#AA000000"
        android:padding="10dp"
        android:text="21/Aug/2017"
        android:textColor="#FFFFFF"
        android:textSize="18sp" />
</FrameLayout>
```

If you observe above code we used **ImageView** to show the image (**flimg**) from drawable folder in framelayout. So add your image to **drawable** folder and replace **@drawable/flimg** path with your image path.

Once we are done with the creation of layout, we need to load the XML layout resource from our **activity onCreate()** callback method, for that open main activity file **MainActivity.java** from **\java\com.tutlane.framelayout** path and write the code like as shown below.

MainActivity.java

```
package com.tutlane.linearlayout;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
}
```

If you observe above code, we are calling our layout using **setContentView** method in the form of **R.layout.layout_file_name**. Here our xml file name is **activity_main.xml** so we used file name **activity_main**.

Generally, during the launch of our activity, the **onCreate()** callback method will be called by the android framework to get the required layout for an activity.

Output of Android FrameLayout Example

When we run the above example using the android virtual device (AVD) we will get a result like as shown below.



Android GridView Example

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/gridview"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:columnWidth="110dp"
    android:numColumns="auto_fit"
    android:verticalSpacing="10dp"
    android:horizontalSpacing="10dp"
    android:stretchMode="columnWidth"
    android:gravity="center" />
```

Once we are done with creation of layout, we need to create a custom adapter (**ImageAdapter.java**) by extending it using **BaseExtender** to show all the items in the grid, for that right click on **java** folder □ Give name as **ImageAdapter.java** and click **OK**.

Open **ImageAdapter.java** file and write the code like as shown below

ImageAdapter.java

```
package com.tutlane.gridview;
import android.content.Context;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.GridView;
import android.widget.ImageView;
/**
 * Created by tutlane on 24-08-2017.
 */
public class ImageAdapter extends BaseAdapter {
    private Context mContext;
    public ImageAdapter(Context c) {
        mContext = c;
    }
    public int getCount() {
        return thumbImages.length;
    }
    public Object getItem(int position) {
        return null;
    }
    public long getItemId(int position) {
        return 0;
    }
    // create a new ImageView for each item referenced by the Adapter
    public View getView(int position, View convertView, ViewGroup parent) {
        ImageView imageView = new ImageView(mContext);
        imageView.setLayoutParams(new GridView.LayoutParams(200, 200));
        imageView.setScaleType(ImageView.ScaleType.CENTER_CROP);
        imageView.setPadding(8, 8, 8, 8);
```

```

        imageView.setImageResource(thumbImages[position]);
        return imageView;
    }
    // Add all our images to arraylist
    public Integer[] thumbImages = {
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5, R.drawable.img6,
        R.drawable.img7, R.drawable.img8,
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5, R.drawable.img6,
        R.drawable.img7, R.drawable.img8,
        R.drawable.img1, R.drawable.img2,
        R.drawable.img3, R.drawable.img4,
        R.drawable.img5
    };
}

```

If you observe above code we referred some images, actually those are the sample images which we added in **/res/drawable** directory.

Now we will bind images to our **GridView** using our custom adapter (**ImageAdapter.java**), for that open main activity file **MainActivity.java** from **\java\com.tutlane.gridview** path and write the code like as shown below.

MainActivity.java

```

package com.tutlane.gridview
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.GridView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        GridView gv = (GridView) findViewById(R.id.gvDetails);
        gv.setAdapter(new ImageAdapter(this));
        gv.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            public void onItemClick(AdapterView<?> parent, View v, int position, long id) {
                Toast.makeText(MainActivity.this, "Image Position: " + position, Toast.LENGTH_SHORT).show();
            }
        });
    }
}

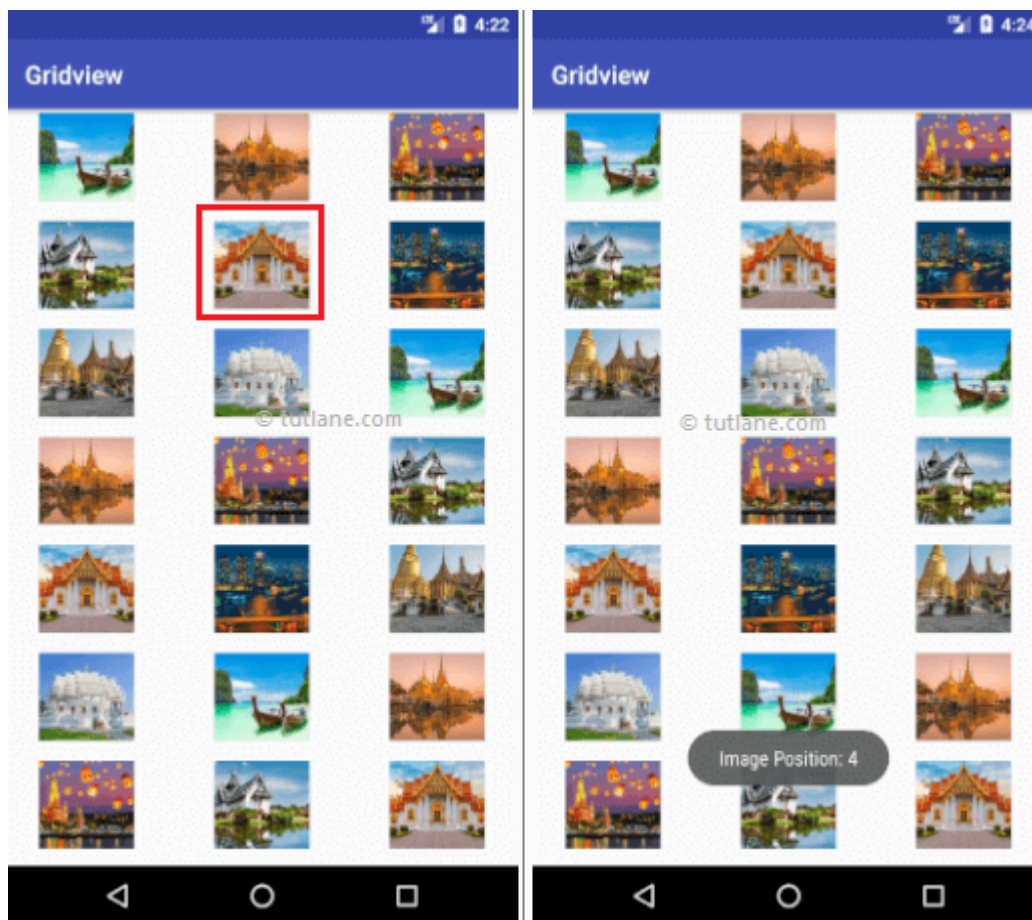
```

If you observe above code, we are binding image details to **GridView** using our custom adapter (**ImageAdapter.java**) and calling our layout using **setContentView** method in the form of **R.layout.layout_file_name**. Here our xml file name is **activity_main.xml** so we used file name **activity_main**.

Generally, during the launch of our activity, **onCreate()** callback method will be called by android framework to get the required layout for an activity.

Output of Android GridView Example

When we run above example using the android virtual device (AVD) we will get a result like as shown below.



This is how we can bind images to GridView using Adapter in android applications based on our requirements.

Android GridView Details Activity Example

image_details.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<ImageView android:id="@+id/full_image_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
</LinearLayout>
```

FullImageActivity.java

```
package com.tutlane.gridview;
import android.app.Activity;
import android.content.Intent;
import android.os.Bundle;
import android.widget.ImageView;

/**
 * Created by tutlane on 24-08-2017.
 */
public class FullImageActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.image_details);
        // Get intent data
        Intent i = getIntent();
        // Get Selected Image Id
        int position = i.getExtras().getInt("id");
        ImageAdapter imageAdapter = new ImageAdapter(this);
        ImageView imageView = (ImageView) findViewById(R.id.full_image_view);
        imageView.setImageResource(imageAdapter.thumbImages[position]);
    }
}
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.tutlane.gridview">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

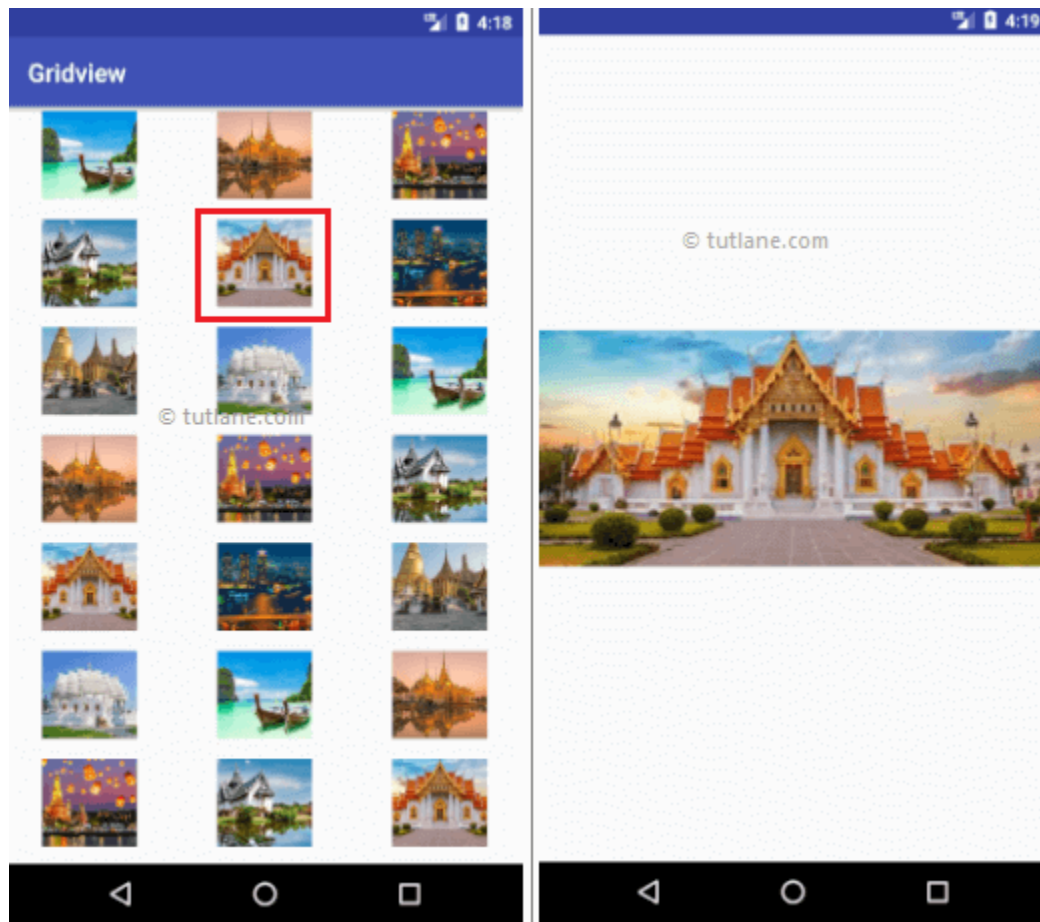
```
</activity>
<!-- FullImageActivity -->
<activity android:name=".FullImageActivity"></activity>
</application>
</manifest>
```

MainActivity.java

```
package com.tutlane.gridview;
import android.content.Intent;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.GridView;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        GridView gv = (GridView) findViewById(R.id.gvDetails);
        gv.setAdapter(new ImageAdapter(this));
        gv.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            public void onItemClick(AdapterView<?> parent, View v, int position, long id) {
                // Sending image id to FullScreenActivity
                Intent i = new Intent(getApplicationContext(), FullImageActivity.class);
                // passing array index
                i.putExtra("id", position);
                startActivity(i);
            }
        });
    }
}
```

Output :-



adding Constraint Layout in Android

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<androidx.constraintlayout.widget.ConstraintLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context=".MainActivity">
```

```
<TextView
```

```
android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginLeft="16dp"
android:layout_marginRight="16dp"
android:gravity="center"
android:padding="10dp"
android:text="Geeks for Geeks"
android:textColor="@color/black"
android:textSize="20sp"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
app:layout_constraintRight_toRightOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Output:



Geeks for Geeks

Practical-6

AIM :- Design various Activities using different Layouts and available Widgets(TextView, EditText, Button, RadioButton, CheckBox, ImageButton, ToggleButton, TimePicker, DatePicker, ProgressBar, ImageView) to make the user-friendly GUI.

Add a Title to the Layout

```
<resources>

    <string name="app_name">LEXO</string>

    <string name="title">Membership Form</string>

</resources>
```

The XML for the TextView should look like this:

```
<TextView

    android:id="@+id/title"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:layout_marginTop="28dp"

    android:text="@string/title"

    android:textColor="#D500F9"

    android:textSize="28sp"

    app:layout_constraintEnd_toEndOf="parent"

    app:layout_constraintHorizontal_bias="0.436"

    app:layout_constraintStart_toStartOf="parent"

    app:layout_constraintTop_toTopOf="parent" />
```

Add a Full Name Field

```
<EditText

    android:id="@+id/names"
```

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="96dp"
android:ems="10"
android:hint="@string/name"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.184"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add the Gender Controls

<TextView

```
android:id="@+id/gender"
android:layout_width="83dp"
android:layout_height="32dp"
android:layout_marginTop="172dp"
android:text="Gender"
android:textSize="18sp"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.118"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

```
android:id="@+id/radioGroup"
android:layout_width="205dp"
android:layout_height="32dp"
android:layout_marginTop="172dp"
```

```
android:orientation="horizontal"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.834"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent">
<RadioButton
    android:id="@+id/radioButton"
    android:layout_width="45dp"
    android:layout_height="33dp"
    android:text="M"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:onClick="radioButtonhandler"/>
<RadioButton
    android:id="@+id/female"
    android:layout_width="45dp"
    android:layout_height="33dp"
    android:text="F"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.655"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:onClick="radioButtonhandler"/>
<RadioButton
    android:id="@+id/other"
```

```
        android:layout_width="96dp"
        android:layout_height="33dp"
        android:text="Other"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.949"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:onClick="radioButtonhandler"/>
    </RadioGroup>
```

Add the Current Weight and Height Fields

```
    <EditText
        android:id="@+id/current_weight"
        android:layout_width="164dp"
        android:layout_height="46dp"
        android:layout_marginTop="260dp"
        android:ems="10"
        android:hint="@string/current_weight"
        android:inputType="number"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.151"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <EditText
        android:id="@+id/height"
        android:layout_width="157dp"
```

```
android:layout_height="44dp"
android:layout_marginTop="260dp"
android:ems="10"
android:hint="@string/height"
android:inputType="number"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.919"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add Goal Weight and Age Fields

<EditText

```
android:id="@+id/goal_weight"
android:layout_width="175dp"
android:layout_height="52dp"
android:layout_marginTop="344dp"
android:ems="10"
android:hint="@string/goal_weight"
android:inputType="number"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.156"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

<EditText

```
android:id="@+id/age"
android:layout_width="140dp"
```

```
android:layout_height="48dp"
android:layout_marginTop="348dp"
android:ems="10"
android:hint="Age"
android:inputType="number"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.863"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add a Phone Number Field

<EditText

```
android:id="@+id/Phone"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="428dp"
android:ems="10"
android:hint="@string/phone"
android:inputType="phone"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.194"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add an Address Field to the Layout

<EditText

```
android:id="@+id/address"
```

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="500dp"
android:ems="10"
android:hint="@string/address"
android:inputType="text"
android:maxLines="2"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.194"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add an "Accept Membership Rules" Checkbox

<CheckBox

```
android:id="@+id/conditions"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="580dp"
android:backgroundTint="#D500F9"
android:buttonTint="#D500F9"
android:text="@string/conditions"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.964"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
```

Add a Submit Button

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="684dp"
    android:backgroundTint="#D500F9"
    android:onClick="submitButtonHandler"
    android:text="SUBMIT"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.425"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Reading User Input

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        EditText nameEditText = (EditText) findViewById(R.id.names);
        String fullName = nameEditText.getText().toString();
    }
```

Responding to Radio Button Click Events

```
<RadioGroup
    android:id="@+id/radioGroup"
    android:layout_width="205dp"
```

```
android:layout_height="32dp"
android:layout_marginTop="172dp"
android:orientation="horizontal"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintHorizontal_bias="0.834"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent">
<RadioButton
    android:id="@+id/radioButton"
    android:layout_width="45dp"
    android:layout_height="33dp"
    android:text="M"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:onClick="radioButtonhandler"/>
<RadioButton
    android:id="@+id/female"
    android:layout_width="45dp"
    android:layout_height="33dp"
    android:text="F"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.655"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:onClick="radioButtonhandler"/>
```

```
<RadioButton  
    android:id="@+id/other"  
    android:layout_width="96dp"  
    android:layout_height="33dp"  
    android:text="Other"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintHorizontal_bias="0.949"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent"  
    android:onClick="radioButtonhandler" />  
</RadioGroup>
```

Implement the method in the Java file.

```
public void radioButtonhandler(View view) {  
    // Decide what happens when a user clicks on a button  
}  
  
c class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        EditText nameEditText = (EditText) findViewById(R.id.names);  
        String fullName = nameEditText.getText().toString();  
        EditText currentWeightEditText = (EditText) findViewById(R.id.current_weight);  
        String currentWeight = currentWeightEditText.getText().toString();  
        EditText heightEditText = (EditText) findViewById(R.id.height);
```

```
String Height = heightEditText.getText().toString();

EditText goalWeightEditText = (EditText) findViewById(R.id.gol_weight);

String GoalWeight = goalWeightEditText.getText().toString();

EditText ageEditText = (EditText) findViewById(R.id.age);

String age = ageEditText.getText().toString();

EditText phoneEditText = (EditText) findViewById(R.id.names);

String phone = phoneEditText.getText().toString();

EditText addressEditText = (EditText) findViewById(R.id.names);

String address = addressEditText.getText().toString();

}
```

Read Input From a Checkbox

```
public void submitbuttonHandler(View view) {

    //Decide what happens when the user clicks the submit button

}
```

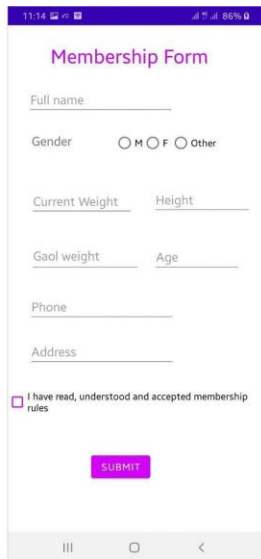
Listening to Button Click Events

```
public void submitbuttonHandler(View view) {

    //Decide what happens when the user clicks the submit button

}
```

OUTPUT



The screenshot displays a mobile application interface with a purple header bar. The status bar at the top shows the time as 11:14, signal strength, and 86% battery. The app title "Membership Form" is centered in the header. The form contains several input fields: "Full name", "Gender" (with radio buttons for M, F, and Other), "Current Weight", "Height", "Goal weight", "Age", "Phone", and "Address". Below these fields is a checkbox labeled "I have read, understood and accepted membership rules". A purple "SUBMIT" button is positioned at the bottom of the form. The mobile OS navigation bar at the very bottom shows three icons: a square, a circle, and a triangle.

PRACTICAL-7

AIM:- Develop code to demonstrate different ways of Handling different events (onClick, onLongClick etc.) over Button, EditText etc. to perform action in Android application at run-time.

Event Listeners	Description
onClick()	The onClick() listener is invoked when the user presses or clicks on an element.
onLongClick()	The onLongClick() listener is triggered when the user long presses an element.
onFocusChange()	The onFocusChange() method is invoked when an element comes in focus or goes out of focus.
onKey()	The onKey() method is triggered when a user presses or releases a key from his keyboard.
onTouch()	The onTouch() method is triggered whenever an element is touched or simply tapped.
onMenuItemClick()	The onMenuItemClick() method is triggered whenever the user clicks on any object in the menu list.
onCreateContextMenu()	The onCreateContextMenu() method is triggered whenever the menu is successfully built in the context or activity.

@Override

```
protected void onCreate(Bundle savedInstanceState) {
    Button button = (Button)findViewById(R.id.btn1);

    button.setOnTouchListener(buttonListener);
// code
}
// Anonymous implementation of OnTouchListener
private View.OnTouchListener buttonListener = new View.OnTouchListener() {
    public void onTouchEvent(View v) {
        // define the action
    }
};
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
```

```
<TextView
    android:id="@+id/textView2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_marginLeft="100dp"
    android:fontFamily="@font/arya_bold"
    android:text="DataFlair "
    android:textColor="@color/colorPrimaryDark"
    android:textSize="50dp" />
```

```
<Button
    android:id="@+id/btnClick"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="150dp"
    android:layout_marginTop="100dp"
    android:fontFamily="@font/arya_bold"
    android:text="Click Event" />
```

```
<TextView
    android:id="@+id/txtResult"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textColor="#009999"
    android:layout_marginLeft="150dp"
    android:textSize="20dp"
    android:textStyle="bold"
    android:layout_marginTop="12dp"/>
```

```
<TextView
    android:id="@+id/textView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_below="@+id/button2"
    android:layout_centerHorizontal="true"
    android:fontFamily="@font/arya_bold"
    android:gravity="center"
    android:text="See me!"
    android:textColor="#009999"
    android:textSize="25dp" />
</LinearLayout>
```

main_activity.java

```
package com.DataFlair.dataflaireventhandling;
```

```
import android.os.Bundle;

import android.view.Gravity;

import android.view.View;

import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;

import android.widget.TextView;

public class MainActivity extends AppCompatActivity {

    Button btn;

    TextView tView;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        btn = (Button)findViewById(R.id.btnClick);

        tView = (TextView)findViewById(R.id.txtResult);

        btn.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {

                TextView txtView = (TextView) findViewById(R.id.textView);

                txtView.setText("You've Clicked \n The Event has taken place");

                txtView.setTextSize(25);

                txtView.setGravity(Gravity.CENTER);

                txtView.setTextColor(R.color.colorAccent);

            }

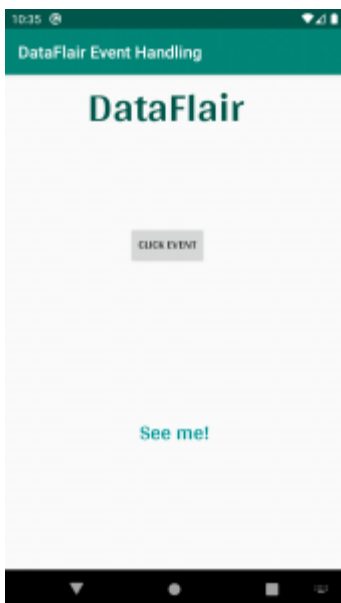
        });

    }

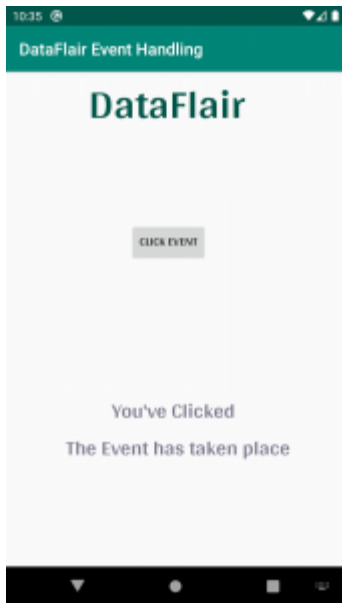
}
```

```
}  
  
<?xml version="1.0" encoding="utf-8"?>  
  
<resources>  
  
    <color name="colorPrimary">#008577</color>  
  
    <color name="colorPrimaryDark">#00574B</color>  
  
    <color name="colorAccent">#D81B60</color>  
  
</resources>
```

OUTPUT:-



Now, we will click and see how the `onClick()` method reacts when we click.



PRACTICAL-8

AIM:- Develop code to demonstrate Event handling of CheckBox and RadioButton selection.

Check Boxes

//In initialization code:

```
chinButton = new JCheckBox("Chin");  
chinButton.setMnemonic(KeyEvent.VK_C);  
chinButton.setSelected(true);  
glassesButton = new JCheckBox("Glasses");  
glassesButton.setMnemonic(KeyEvent.VK_G);  
glassesButton.setSelected(true);  
hairButton = new JCheckBox("Hair");  
hairButton.setMnemonic(KeyEvent.VK_H);  
hairButton.setSelected(true);
```

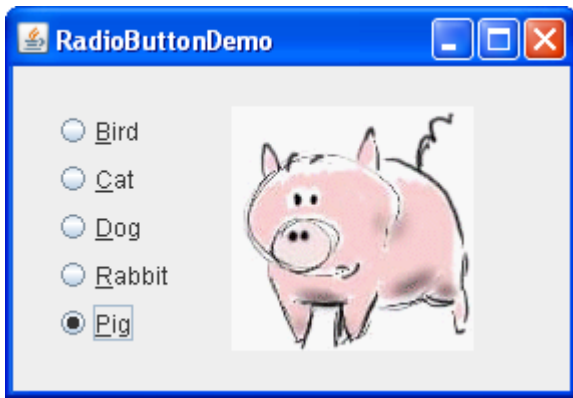
```
teethButton = new JCheckBox("Teeth");  
teethButton.setMnemonic(KeyEvent.VK_T);  
teethButton.setSelected(true);
```

//Register a listener for the check boxes.

```
chinButton.addItemListener(this);  
glassesButton.addItemListener(this);  
hairButton.addItemListener(this);  
teethButton.addItemListener(this);
```

...

```
public void itemStateChanged(ItemEvent e) {  
  
    ...  
  
    Object source = e.getItemSelectable();  
  
    if (source == chinButton) {  
        //...make a note of it...  
    } else if (source == glassesButton) {  
        //...make a note of it...  
    } else if (source == hairButton) {  
        //...make a note of it...  
    } else if (source == teethButton) {  
        //...make a note of it...  
    }  
  
    if (e.getStateChange() == ItemEvent.DESELECTED)  
        //...make a note of it...  
  
    ...  
  
    updatePicture();  
}
```

OUTPUT:-

//In initialization code:

//Create the radio buttons.

```
JRadioButton birdButton = new JRadioButton(birdString);
```

```
birdButton.setMnemonic(KeyEvent.VK_B);
```

```
birdButton.setActionCommand(birdString);
```

```
birdButton.setSelected(true);
```

```
JRadioButton catButton = new JRadioButton(catString);
```

```
catButton.setMnemonic(KeyEvent.VK_C);
```

```
catButton.setActionCommand(catString);
```

```
JRadioButton dogButton = new JRadioButton(dogString);
```

```
dogButton.setMnemonic(KeyEvent.VK_D);
```

```
dogButton.setActionCommand(dogString);
```

```
JRadioButton rabbitButton = new JRadioButton(rabbitString);
```

```
rabbitButton.setMnemonic(KeyEvent.VK_R);
```

```
rabbitButton.setActionCommand(rabbitString);
```

```
JRadioButton pigButton = new JRadioButton(pigString);

pigButton.setMnemonic(KeyEvent.VK_P);

pigButton.setActionCommand(pigString);


//Group the radio buttons.

ButtonGroup group = new ButtonGroup();

group.add(birdButton);

group.add(catButton);

group.add(dogButton);

group.add(rabbitButton);

group.add(pigButton);


//Register a listener for the radio buttons.

birdButton.addActionListener(this);

catButton.addActionListener(this);

dogButton.addActionListener(this);

rabbitButton.addActionListener(this);

pigButton.addActionListener(this);

...

public void actionPerformed(ActionEvent e) {

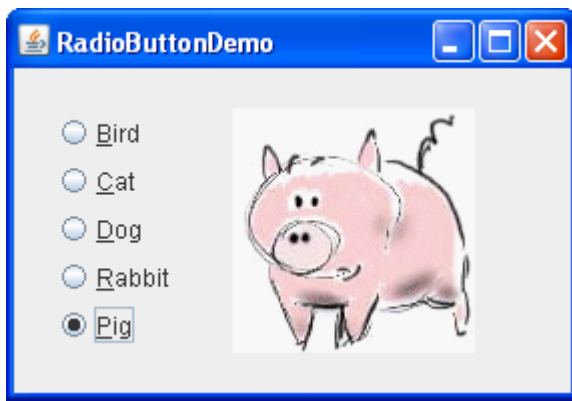
    picture.setIcon(new ImageIcon("images/"

        + e.getActionCommand()

        + ".gif"));

}
```

OUTPUT:-



PRACTICAL-9

AIM:- Develop code to navigate between different activities and pass the data from one activity to other activity using Intent.

➤ **FIRST XML FILE**

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    tools:context=".first_activity">
```

```
    <EditText
```

```
        android:id="@+id/send_text_id"
```

```
        android:layout_width="300dp"
```

```
        android:layout_height="wrap_content"
```

```
        android:layout_marginLeft="40dp"
```

```
        android:layout_marginTop="20dp"
```

```
        android:hint="Input"
```

```
        android:textSize="25dp"
```

```
        android:textStyle="bold" />
```

```
    <Button
```

```
        android:id="@+id/send_button_id"
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="40dp"
```

```
        android:layout_marginLeft="150dp"
```

```
        android:layout_marginTop="150dp"
```

```
        android:text="send"

        android:textStyle="bold" />

</RelativeLayout>
```

➤ **FIRST JAVA FILE**

```
import android.content.Intent;

import android.os.Bundle;

import android.widget.Button;

import android.widget.EditText;

import androidx.appcompat.app.AppCompatActivity;
```

```
public class first_activity extends AppCompatActivity {
```

```
    // define the variable
```

```
    Button send_button;
```

```
    EditText send_text;
```

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.activity_first_activity);
```

```
    send_button = findViewById(R.id.send_button_id);
```

```
    send_text = findViewById(R.id.send_text_id);
```

```
    // add the OnClickListener in sender button after clicked this button following Instruction will
```

```
run
```

```
    send_button.setOnClickListener(v -> {
```

```
// get the value which input by user in EditText and convert it to string
String str = send_text.getText().toString();

// Create the Intent object of this class Context() to Second_activity class
Intent intent = new Intent(getApplicationContext(), Second_activity.class);

// now by putExtra method put the value in key, value pair key is
// message_key by this key we will receive the value, and put the string
intent.putExtra("message_key", str);

// start the Intent
startActivity(intent);

});

}
```

➤ SECOND XML FILE

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="org.geeksforgeeks.navedmalik.sendthedata.Second_activity">

    <TextView
        android:id="@+id/received_value_id"
        android:layout_width="300dp"
        android:layout_height="50dp"
        android:layout_marginLeft="40dp"
        android:layout_marginTop="20dp"
```

```
        android:textSize="40sp"
        android:textStyle="bold"
        android:layout_marginStart="40dp" />
</RelativeLayout>
```

➤ SECOND JAVA FILE

```
import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

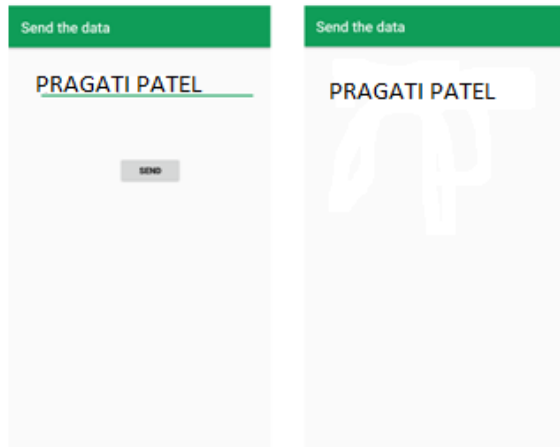
public class Second_activity extends AppCompatActivity {

    TextView receiver_msg;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second_activity);

        receiver_msg = findViewById(R.id.received_value_id);
        // create the get Intent object
        Intent intent = getIntent();
        // receive the value by getStringExtra() method and
        // key must be same which is send by first activity
        String str = intent.getStringExtra("message_key");
        // display the string into textView
```

```
        receiver_msg.setText(str);  
    }  
}
```

Output:

PRACTICAL-10

AIM:- Develop an android application to store data locally using SharedPreferences and access-modify in different activities.

MainActivity.java.

```
package com.example.sairamkrishna.myapplication;

import android.content.Context;

import android.content.SharedPreferences;

import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.Button;

import android.widget.EditText;

import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    EditText ed1,ed2,ed3;

    Button b1;

    public static final String MyPREFERENCES = "MyPrefs" ;

    public static final String Name = "nameKey";

    public static final String Phone = "phoneKey";

    public static final String Email = "emailKey";

    SharedPreferences sharedPreferences;
```

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
  
    super.onCreate(savedInstanceState);  
  
    setContentView(R.layout.activity_main);  
  
  
    ed1=(EditText)findViewById(R.id.editText);  
    ed2=(EditText)findViewById(R.id.editText2);  
    ed3=(EditText)findViewById(R.id.editText3);  
  
  
    b1=(Button)findViewById(R.id.button);  
  
    sharedPreferences = getSharedPreferences(MyPREFERENCES, Context.MODE_PRIVATE);  
  
  
    b1.setOnClickListener(new View.OnClickListener() {  
  
        @Override  
  
        public void onClick(View v) {  
  
            String n = ed1.getText().toString();  
  
            String ph = ed2.getText().toString();  
  
            String e = ed3.getText().toString();  
  
  
            SharedPreferences.Editor editor = sharedPreferences.edit();  
  
  
            editor.putString(Name, n);  
  
            editor.putString(Phone, ph);  
  
            editor.putString(Email, e);  

```

```
        editor.commit();

        Toast.makeText(MainActivity.this,"Thanks",Toast.LENGTH_LONG).show();

    }

});

}

}
```

activity_main.xml.

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools" android:layout_width="match_parent"

    android:layout_height="match_parent" android:paddingLeft="@dimen/activity_horizontal_margin"

    android:paddingRight="@dimen/activity_horizontal_margin"

    android:paddingTop="@dimen/activity_vertical_margin"

    android:paddingBottom="@dimen/activity_vertical_margin" tools:context=".MainActivity">

    <TextView

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Shared Preference "

        android:id="@+id/textView"

        android:layout_alignParentTop="true"

        android:layout_centerHorizontal="true"

        android:textSize="35dp" />
```

```
<TextView  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Tutorials Point"  
    android:id="@+id/textView2"  
    android:layout_below="@+id/textView"  
    android:layout_centerHorizontal="true"  
    android:textSize="35dp"  
    android:textColor="#ff16ff01" />
```

```
<EditText  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:id="@+id/editText"  
    android:layout_below="@+id/textView2"  
    android:layout_marginTop="67dp"  
    android:hint="Name"  
    android:layout_alignParentRight="true"  
    android:layout_alignParentEnd="true"  
    android:layout_alignParentLeft="true"  
    android:layout_alignParentStart="true" />
```

```
<EditText  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"
```

```
android:id="@+id/editText2"
android:layout_below="@+id/editText"
android:layout_alignParentLeft="true"
android:layout_alignParentStart="true"
android:layout_alignParentRight="true"
android:layout_alignParentEnd="true"
android:hint="Pass" />
```

```
<EditText
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/editText3"
    android:layout_below="@+id/editText2"
    android:layout_alignParentLeft="true"
    android:layout_alignParentStart="true"
    android:layout_alignParentRight="true"
    android:layout_alignParentEnd="true"
    android:hint="Email" />
```

```
<Button
```

```
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Save"
    android:id="@+id/button"
    android:layout_below="@+id/editText3"
    android:layout_centerHorizontal="true"
```

```
    android:layout_marginTop="50dp" />
```

```
</RelativeLayout>
```

res/values/strings.xml.

```
<resources>
```

```
    <string name="app_name">My Application</string>
```

```
</resources>
```

AndroidManifest.xml.

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    package="com.example.sairamkrishna.myapplication" >
```

```
    <application
```

```
        android:allowBackup="true"
```

```
        android:icon="@mipmap/ic_launcher"
```

```
        android:label="@string/app_name"
```

```
        android:theme="@style/AppTheme" >
```

```
        <activity
```

```
            android:name=".MainActivity"
```

```
            android:label="@string/app_name" >
```

```
                <intent-filter>
```

```
                    <action android:name="android.intent.action.MAIN" />
```

```
                    <category android:name="android.intent.category.LAUNCHER" />
```

```
</intent-filter>
```

```
</activity>
```

```
</application>
```

```
</manifest>
```

OUTPUT:-



PRACTICAL-11

AIM:- Develop the code to implement the ListView and the Spinner views, perform add, update, remove items operations and implement the item selection event handling over ListView and Spinner for appropriate example.

Android Spinner Example

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="example.javatpoint.com.spinner.MainActivity">

    <Spinner
        android:id="@+id/spinner"
        android:layout_width="149dp"
        android:layout_height="40dp"
        android:layout_marginBottom="8dp"
        android:layout_marginEnd="8dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.502"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
```

```
app:layout_constraintVertical_bias="0.498" />
```

```
</android.support.constraint.ConstraintLayout>
```

Activity class

Let's write the code to display item on the spinner and perform event handling.

File: MainActivity.java

```
package example.javatpoint.com.spinner;
```

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.view.View;
```

```
import android.widget.AdapterView;
```

```
import android.widget.AdapterView;
```

```
import android.widget.AdapterView;
```

```
import android.widget.AdapterView;
```

```
public class MainActivity extends AppCompatActivity implements
```

```
    AdapterView.OnItemClickListener {
```

```
    String[] country = { "India", "USA", "China", "Japan", "Other"};
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
        //Getting the instance of Spinner and applying OnItemSelectedListener on it
```

```
Spinner spin = (Spinner) findViewById(R.id.spinner);

spin.setOnItemSelectedListener(this);

//Creating the ArrayAdapter instance having the country list

ArrayAdapter aa = new ArrayAdapter(this,android.R.layout.simple_spinner_item,country);
aa.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);

//Setting the ArrayAdapter data on the Spinner

spin.setAdapter(aa);
}

//Performing action onItemSelected and onNothing selected

@Override

public void onItemSelected(AdapterView<?> arg0, View arg1, int position, long id) {

    Toast.makeText(getApplicationContext(),country[position] , Toast.LENGTH_LONG).show();

}

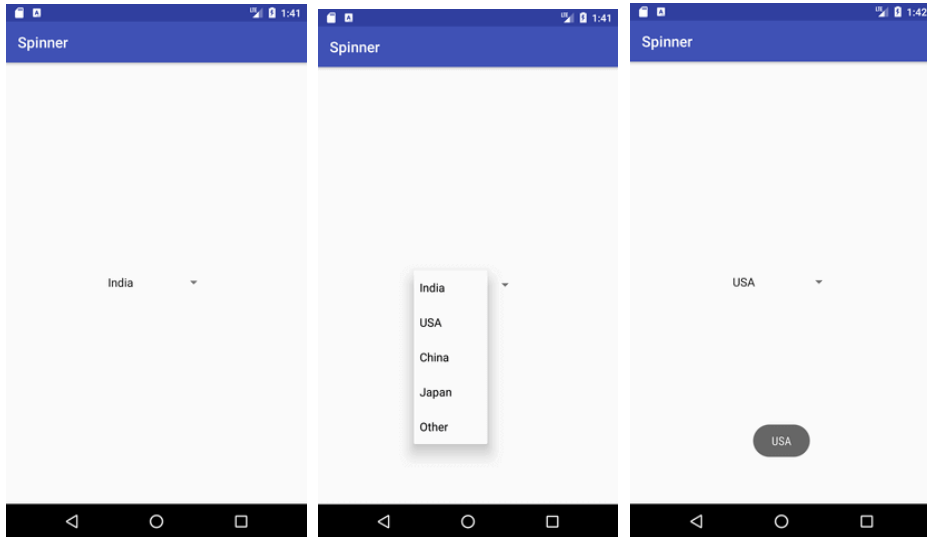
@Override

public void onNothingSelected(AdapterView<?> arg0) {

    // TODO Auto-generated method stub

}

}
```

Output:**ACTIVITY_MAIN.XML FILE FOR LIST VIEW**

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<RelativeLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:id="@+id/idRLContainer"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    tools:context=".MainActivity">
```

```
<!--on below line we are creating edit text
```

```
    for adding a new item to list-->
```

```
<EditText
```

```
    android:id="@+id/idEdtItemName"
```

```
    android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:layout_margin="4dp"
        android:layout_toLeftOf="@id/idBtnAdd"
        android:hint="Enter item name to add in list" />
```

```
<!--button for adding item from edit text to list-->
```

```
<Button
```

```
    android:id="@+id/idBtnAdd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentEnd="true"
    android:layout_margin="4dp"
    android:text="Add"
    android:textAllCaps="false" />
```

```
<!--list view to display list of languages-->
```

```
<ListView
```

```
    android:id="@+id/idLVLanguages"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_below="@id/idEdtItemName" />
```

```
</RelativeLayout>
```

MainActivity file

```
package com.gtappdevelopers.kotlingfgproject
```

```
import android.os.Bundle

import android.widget.AdapterView

import android.widget.Button

import android.widget.EditText

import android.widget.ListView

import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    // on below line we are creating a variable.

    lateinit var languageLV: ListView

    lateinit var addBtn: Button

    lateinit var itemEdt: EditText

    lateinit var lngList: ArrayList<String>

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        // on below line we are initializing our variables.

        languageLV = findViewById(R.id.idLVLanguages)

        addBtn = findViewById(R.id.idBtnAdd)

        itemEdt = findViewById(R.id.idEdtItemName)

        lngList = ArrayList()
```

```
// on below line we are adding items to our list

lngList.add("C++")

lngList.add("Python")


// on below line we are initializing adapter for our list view.
val adapter: ArrayAdapter<String?> = ArrayAdapter<String?>(
    this@MainActivity,
    android.R.layout.simple_list_item_1,
    lngList as List<String?>
)


// on below line we are setting adapter for our list view.
languageLV.adapter = adapter


// on below line we are adding click listener for our button.
addBtn.setOnClickListener {

    // on below line we are getting text from edit text
    val item = itemEdt.text.toString()

    // on below line we are checking if item is not empty
    if (item.isNotEmpty()) {
        // on below line we are adding item to our list.
        lngList.add(item)

        // on below line we are notifying adapter
```

```
// that data in list is updated to update our list view.
```

```
adapter.notifyDataSetChanged()
```

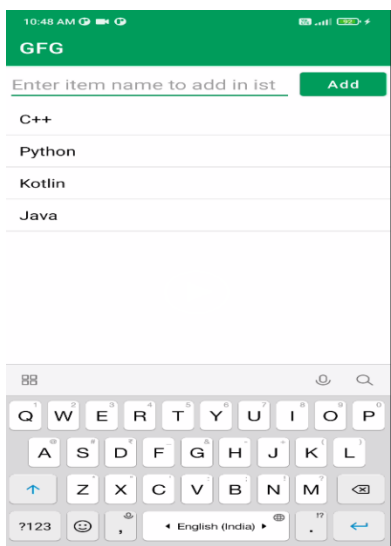
```
}
```

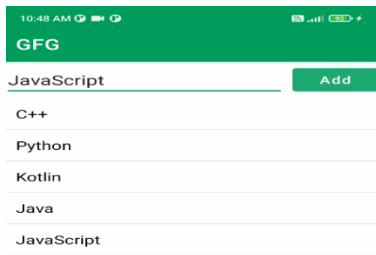
```
}
```

```
}
```

```
}
```

OUTPUT:-





PRACTICAL-12

AIM:- Develop the code to manage Permission using Manifest file and run time from Activity, and toggle state of WiFi and Bluetooth.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>

<manifest

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:versionCode="1"

    android:versionName="1.0">

    <!-- Beware that these values are overridden by the build.gradle file -->

    <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="26" />

    <application

        android:allowBackup="true"

        android:icon="@mipmap/ic_launcher"

        android:roundIcon="@mipmap/ic_launcher_round"

        android:label="@string/app_name"

        android:supportsRtl="true"

        android:theme="@style/AppTheme">

        <!-- This name is resolved to com.example.myapp.MainActivity

            based on the namespace property in the build.gradle file -->

        <activity android:name=".MainActivity">

            <intent-filter>

                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />

            </intent-filter>

        </activity>

    </application>

</manifest>
```

```

        </intent-filter>

    </activity>

    <activity

        android:name=".DisplayMessageActivity"

        android:parentActivityName=".MainActivity" />

</application>

</manifest>

```

manifest code:

```

<!--BLUETOOTH PERMISSION-->
<!-- Request legacy Bluetooth permissions on older devices. -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<!-- Needed only if your app looks for Bluetooth devices.
     If your app doesn't use Bluetooth scan results to derive physical
     location information, you can strongly assert that your app
     doesn't derive physical location. -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<!-- Needed only if your app makes the device discoverable to Bluetooth
     devices. -->
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
<!-- Needed only if your app communicates with already-paired Bluetooth
     devices. -->
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<!--bibo01 : hardware option-->
<uses-feature android:name="android.hardware.bluetooth" android:required="false"/>
<uses-feature android:name="android.hardware.bluetooth_le" android:required="false"/>

```

Kotlin code: //check android12+

```

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    requestMultiplePermissions.launch(arrayOf(
        Manifest.permission.BLUETOOTH_SCAN,
        Manifest.permission.BLUETOOTH_CONNECT))
}
else{
    val enableBtIntent = Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE)
    requestBluetooth.launch(enableBtIntent)
}

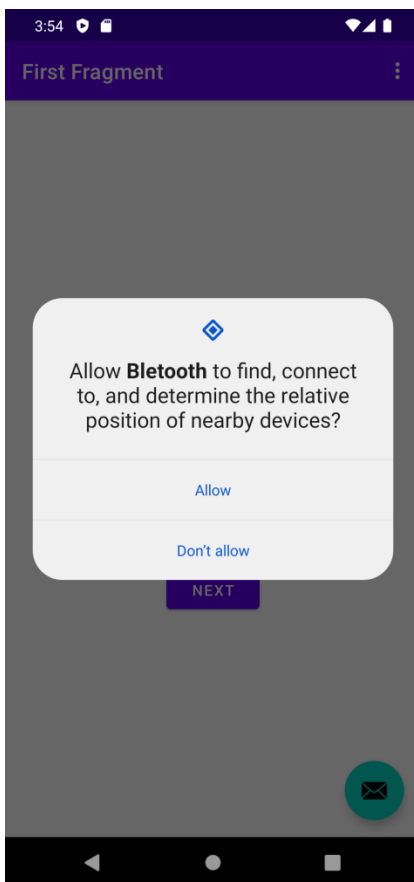
```

.....

```
private var requestBluetooth = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) {
    result ->
        if (result.resultCode == RESULT_OK) {
            //granted
        } else {
            //deny
        }
}

private val requestMultiplePermissions =
    registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) { permissions ->
        permissions.entries.forEach {
            Log.d("test006", "${it.key} = ${it.value}")
        }
    }
```

OUTPUT:-



PRACTICAL-13

AIM:- Develop android applications to demonstrate user interaction with the application using Options Menu, Context Menu and Popup Menu.

XML

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity">

    <Button

        android:id="@+id/clickBtn"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:background="#0F9D58"

        android:text="Click Me"

        android:textColor="#ffffff"

        app:layout_constraintBottom_toBottomOf="parent"

        app:layout_constraintLeft_toLeftOf="parent"

        app:layout_constraintRight_toRightOf="parent"

        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

```
<resources>

    <color name="colorPrimary">#0F9D58</color>

    <color name="colorPrimaryDark">#16E37F</color>

    <color name="colorAccent">#03DAC5</color>

</resources>

<?xml version="1.0" encoding="utf-8"?>

<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item

        android:id="@+id/java"

        android:title="Java" />

    <item

        android:id="@+id/kotlin"

        android:title="Kotlin" />

    <item

        android:id="@+id/android"

        android:title="Android" />

    <item

        android:id="@+id/react_native"

        android:title="React Native" />

</menu>
```

JAVA

```
import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;

import android.view.MenuItem;

import android.view.View;

import android.widget.Button;

import android.widget.PopupMenu;

import android.widget.Toast;


public class MainActivity extends AppCompatActivity {

    Button button;


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);


        // Referencing and Initializing the button

        button = (Button) findViewById(R.id.clickBtn);


        // Setting onClick behavior to the button

        button.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View view) {

                // Initializing the popup menu and giving the reference as current context
```

```
        PopupMenu popupMenu = new PopupMenu(MainActivity.this, button);

        // Inflating popup menu from popup_menu.xml file
        popupMenu.getMenuInflater().inflate(R.menu.popup_menu,
popupMenu.getMenu());

        popupMenu.setOnMenuItemClickListener(new
PopupMenu.OnMenuItemClickListener() {

            @Override

            public boolean onMenuItemClick(MenuItem menuItem) {

                // Toast message on menu item clicked

                Toast.makeText(MainActivity.this, "You Clicked " +
menuItem.getTitle(), Toast.LENGTH_SHORT).show();

                return true;

            }

        });

        // Showing the popup menu
        popupMenu.show();

    }

});

}

import android.os.Bundle

import android.widget.Button

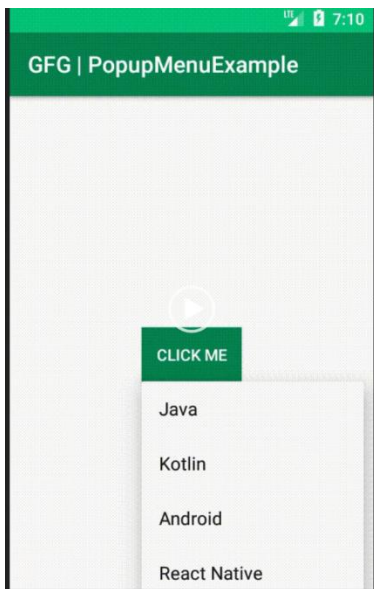
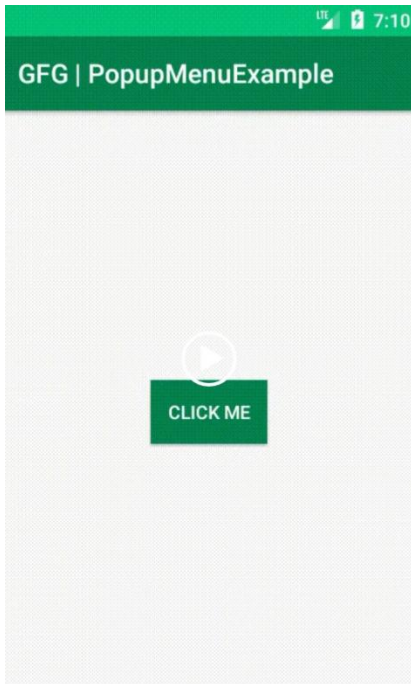
import android.widget.PopupMenu

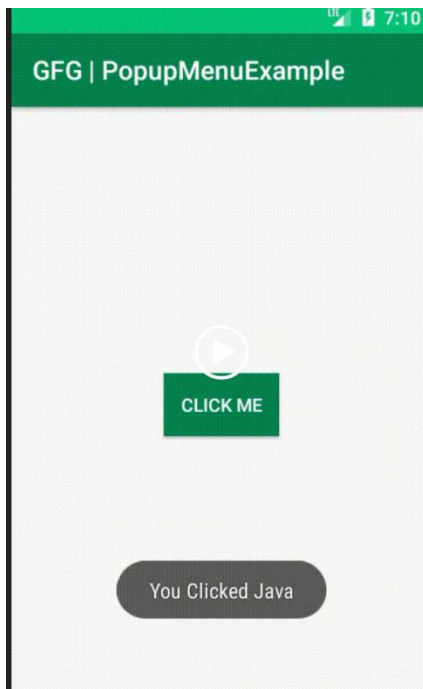
import android.widget.Toast

import androidx.appcompat.app.AppCompatActivity
```

```
class MainActivity : AppCompatActivity() {  
  
    lateinit var button: Button  
  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        // Referencing and Initializing the button  
        button = findViewById(R.id.clickBtn)  
  
        // Setting onClick behavior to the button  
        button.setOnClickListener {  
            // Initializing the popup menu and giving the reference as current context  
            val popupMenu = PopupMenu(this@MainActivity, button)  
  
            // Inflating popup menu from popup_menu.xml file  
            popupMenu.menuInflater.inflate(R.menu.popup_menu, popupMenu.menu)  
            popupMenu.setOnMenuItemClickListener { menuItem ->  
                // Toast message on menu item clicked  
                Toast.makeText(this@MainActivity, "You Clicked " + menuItem.title,  
                    Toast.LENGTH_SHORT).show()  
                true  
            }  
            // Showing the popup menu  
            popupMenu.show()  
        }  
    }  
}
```

```
}
```

OUTPUT:-



PRACTICAL-14

AIM:- Develop Android Applications to demonstrate different AlertDialogs and the Custom Dialog.

custom_layout.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical"

    android:paddingLeft="20dp"

    android:paddingRight="20dp">

    <EditText

        android:id="@+id/editText"

        android:layout_width="match_parent"

        android:layout_height="wrap_content" />

</LinearLayout>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:id="@+id/root"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:gravity="center">
```

```
android:orientation="vertical"
tools:context=".MainActivity">
```

```
<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:onClick="showAlertDialogButtonClicked"
    android:text="Show Dialog" />
</LinearLayout>
```

custom_layout.xml

```
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void showAlertDialogButtonClicked(View view) {
        // Create an alert builder
        AlertDialog.Builder builder = new AlertDialog.Builder(this);
```

```
builder.setTitle("Name");

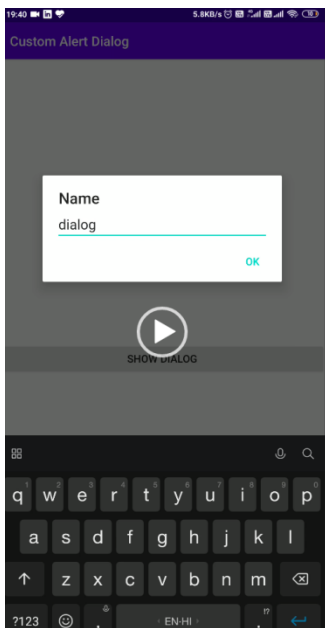
// set the custom layout
final View customLayout = getLayoutInflater().inflate(R.layout.custom_layout, null);
builder.setView(customLayout);

// add a button
builder.setPositiveButton("OK", (dialog, which) -> {
    // send data from the AlertDialog to the Activity
    EditText editText = customLayout.findViewById(R.id.editText);
    sendDialogDataToActivity(editText.getText().toString());
});

// create and show the alert dialog
AlertDialog dialog = builder.create();
dialog.show();
}

// Do something with the data coming from the AlertDialog
private void sendDialogDataToActivity(String data) {
    Toast.makeText(this, data, Toast.LENGTH_SHORT).show();
}
}
```

Output



PRACTICAL-15

AIM:- Develop Android Application for local database connectivity and performing basic database operations (select, insert, update, delete) using SQLiteDatabase and SQLiteOpenHelper Classes

android SQLite database

package example.javatpoint.com.sqlitetutorial;

```
public class Contact {  
    int _id;  
    String _name;  
    String _phone_number;  
    public Contact() { }  
    public Contact(int id, String name, String _phone_number){  
        this._id = id;  
        this._name = name;  
        this._phone_number = _phone_number;  
    }  
  
    public Contact(String name, String _phone_number){  
        this._name = name;  
        this._phone_number = _phone_number;  
    }  
    public int getID(){  
        return this._id;  
    }  
  
    public void setID(int id){  
        this._id = id;
```

```
}

public String getName(){
    return this._name;
}

public void setName(String name){
    this._name = name;
}

public String getPhoneNumber(){
    return this._phone_number;
}

public void setPhoneNumber(String phone_number){
    this._phone_number = phone_number;
}
}
```

File: DatabaseHandler.java

Now, let's create the database handler class that extends SQLiteOpenHelper class and provides the implementation of its methods.

```
package example.javatpoint.com.sqlitetutorial;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
```

```
import android.database.sqlite.SQLiteDatabase;

import android.database.sqlite.SQLiteOpenHelper;

import java.util.ArrayList;

import java.util.List;

public class DatabaseHandler extends SQLiteOpenHelper {

    private static final int DATABASE_VERSION = 1;

    private static final String DATABASE_NAME = "contactsManager";

    private static final String TABLE_CONTACTS = "contacts";

    private static final String KEY_ID = "id";

    private static final String KEY_NAME = "name";

    private static final String KEY_PH_NO = "phone_number";

    public DatabaseHandler(Context context) {

        super(context, DATABASE_NAME, null, DATABASE_VERSION);

        //3rd argument to be passed is CursorFactory instance
    }

    // Creating Tables

    @Override

    public void onCreate(SQLiteDatabase db) {

        String CREATE_CONTACTS_TABLE = "CREATE TABLE " + TABLE_CONTACTS + "("

            + KEY_ID + " INTEGER PRIMARY KEY," + KEY_NAME + " TEXT,"

            + KEY_PH_NO + " TEXT" + ")";

        db.execSQL(CREATE_CONTACTS_TABLE);

    }
```

```
// Upgrading database

@Override

public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

    // Drop older table if existed

    db.execSQL("DROP TABLE IF EXISTS " + TABLE_CONTACTS);


    // Create tables again

    onCreate(db);

}


// code to add the new contact

void addContact(Contact contact) {

    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();

    values.put(KEY_NAME, contact.getName()); // Contact Name

    values.put(KEY_PH_NO, contact.getPhoneNumber()); // Contact Phone


    // Inserting Row

    db.insert(TABLE_CONTACTS, null, values);

    //2nd argument is String containing nullColumnHack

    db.close(); // Closing database connection

}


// code to get the single contact
```

```
Contact getContact(int id) {  
    SQLiteDatabase db = this.getReadableDatabase();  
  
    Cursor cursor = db.query(TABLE_CONTACTS, new String[] { KEY_ID,  
        KEY_NAME, KEY_PH_NO }, KEY_ID + "=?",  
        new String[] { String.valueOf(id) }, null, null, null, null);  
    if (cursor != null)  
        cursor.moveToFirst();  
  
    Contact contact = new Contact(Integer.parseInt(cursor.getString(0)),  
        cursor.getString(1), cursor.getString(2));  
    // return contact  
    return contact;  
}  
  
// code to get all contacts in a list view  
public List<Contact> getAllContacts() {  
    List<Contact> contactList = new ArrayList<Contact>();  
    // Select All Query  
    String selectQuery = "SELECT * FROM " + TABLE_CONTACTS;  
  
    SQLiteDatabase db = this.getWritableDatabase();  
    Cursor cursor = db.rawQuery(selectQuery, null);  
  
    // looping through all rows and adding to list  
    if (cursor.moveToFirst()) {
```

```
do {  
    Contact contact = new Contact();  
    contact.setID(Integer.parseInt(cursor.getString(0)));  
    contact.setName(cursor.getString(1));  
    contact.setPhoneNumber(cursor.getString(2));  
    // Adding contact to list  
    contactList.add(contact);  
} while (cursor.moveToNext());  
}  
  
// return contact list  
return contactList;  
}  
  
// code to update the single contact  
public int updateContact(Contact contact) {  
    SQLiteDatabase db = this.getWritableDatabase();  
  
    ContentValues values = new ContentValues();  
    values.put(KEY_NAME, contact.getName());  
    values.put(KEY_PH_NO, contact.getPhoneNumber());  
  
    // updating row  
    return db.update(TABLE_CONTACTS, values, KEY_ID + " = ?",  
        new String[] { String.valueOf(contact.getID()) });  
}
```

// Deleting single contact

```
public void deleteContact(Contact contact) {  
    SQLiteDatabase db = this.getWritableDatabase();  
    db.delete(TABLE_CONTACTS, KEY_ID + " = ?",  
        new String[] { String.valueOf(contact.getID()) });  
    db.close();  
}
```

// Getting contacts Count

```
public int getContactsCount() {  
    String countQuery = "SELECT * FROM " + TABLE_CONTACTS;  
    SQLiteDatabase db = this.getReadableDatabase();  
    Cursor cursor = db.rawQuery(countQuery, null);  
    cursor.close();  
  
    // return count  
    return cursor.getCount();  
}
```

}

File: MainActivity.java

```
package example.javatpoint.com.sqlitetutorial;
```

```
import android.support.v7.app.AppCompatActivity;
```

```
import android.os.Bundle;
```

```
import android.util.Log;
```

```
import java.util.List;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        DatabaseHandler db = new DatabaseHandler(this);

        // Inserting Contacts

        Log.d("Insert: ", "Inserting ..");

        db.addContact(new Contact("Ravi", "9100000000"));
        db.addContact(new Contact("Srinivas", "9199999999"));
        db.addContact(new Contact("Tommy", "9522222222"));
        db.addContact(new Contact("Karthik", "9533333333"));

        // Reading all contacts

        Log.d("Reading: ", "Reading all contacts..");

        List<Contact> contacts = db.getAllContacts();

        for (Contact cn : contacts) {

            String log = "Id: " + cn.getID() + " ,Name: " + cn.getName() + " ,Phone: " +
                cn.getPhoneNumber();

            // Writing Contacts to log

            Log.d("Name: ", log);
```

```

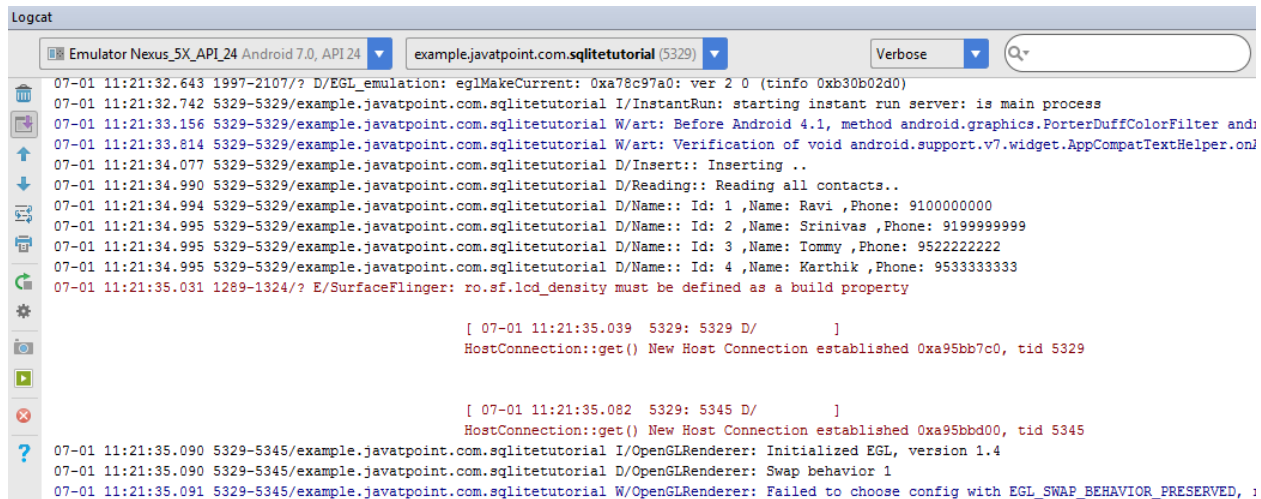
    }

    }

}

```

Output



```

Logcat
Emulator Nexus_5X_API_24 Android 7.0, API 24 example.javatpoint.com.sqlitetutorial (5329) Verbose Q
07-01 11:21:32.643 1997-2107/? D/EGL_emulation: eglMakeCurrent: 0xa78c97a0: ver 2 0 (tinfo 0xb30b02d0)
07-01 11:21:32.742 5329-5329/example.javatpoint.com.sqlitetutorial I/InstantRun: starting instant run server: is main process
07-01 11:21:33.156 5329-5329/example.javatpoint.com.sqlitetutorial W/art: Before Android 4.1, method android.graphics.PorterDuffColorFilter and
07-01 11:21:33.814 5329-5329/example.javatpoint.com.sqlitetutorial W/art: Verification of void android.support.v7.widget.AppCompatTextHelper.onI
07-01 11:21:34.077 5329-5329/example.javatpoint.com.sqlitetutorial D/Insert:: Inserting ..
07-01 11:21:34.990 5329-5329/example.javatpoint.com.sqlitetutorial D/Reading:: Reading all contacts..
07-01 11:21:34.994 5329-5329/example.javatpoint.com.sqlitetutorial D/Name:: Id: 1 ,Name: Ravi ,Phone: 9100000000
07-01 11:21:34.995 5329-5329/example.javatpoint.com.sqlitetutorial D/Name:: Id: 2 ,Name: Srinivas ,Phone: 9199999999
07-01 11:21:34.995 5329-5329/example.javatpoint.com.sqlitetutorial D/Name:: Id: 3 ,Name: Tommy ,Phone: 9522222222
07-01 11:21:34.995 5329-5329/example.javatpoint.com.sqlitetutorial D/Name:: Id: 4 ,Name: Karthik ,Phone: 9533333333
07-01 11:21:35.031 1289-1324/? E/SurfaceFlinger: ro.sf.lcd_density must be defined as a build property

[ 07-01 11:21:35.039 5329: 5329 D/
HostConnection::get() New Host Connection established 0xa95bb7c0, tid 5329

[ 07-01 11:21:35.082 5329: 5345 D/
HostConnection::get() New Host Connection established 0xa95bbd00, tid 5345
07-01 11:21:35.090 5329-5345/example.javatpoint.com.sqlitetutorial I/OpenGLRenderer: Initialized EGL, version 1.4
07-01 11:21:35.090 5329-5345/example.javatpoint.com.sqlitetutorial D/OpenGLRenderer: Swap behavior 1
07-01 11:21:35.091 5329-5345/example.javatpoint.com.sqlitetutorial W/OpenGLRenderer: Failed to choose config with EGL_SWAP_BEHAVIOR_PRESERVED, 1

```

PRACTICAL-16

AIM:- Develop an Android Application to demonstrate the use of RecyclerView and CardView for displaying list of items with multiple information

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity">

    <!--Welcome text-->

    <TextView

        android:id="@+id/heading"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Welcome to GFG!"

        android:textColor="#006600"

        android:textSize="20dp"

        app:layout_constraintBottom_toBottomOf="parent"

        app:layout_constraintEnd_toEndOf="parent"

        app:layout_constraintStart_toStartOf="parent"

        app:layout_constraintTop_toTopOf="parent"

        app:layout_constraintVertical_bias="0.058"></TextView>
```

```
<!-- Main RecyclerView-->

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="15dp"
    android:layout_marginStart="15dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/heading"
    app:layout_constraintVertical_bias="0.061"
    android:paddingBottom="100dp"/>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

layout_one.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <!--parent CardView-->
```

```
<androidx.cardview.widget.CardView
    android:id="@+id/cardview_one"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:layout_marginEnd="16dp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    android:elevation="5dp">

<!--LinearLayout inside the CardView-->

<!--This layout is accessed to create
    toasts when this item is clicked-->

<LinearLayout
    android:id="@+id/linearlayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">

    <!--This layout only holds a TextView
        inside a CardView-->

    <TextView
        android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="#000000"
```

```
        android:textSize="20dp"
        android:layout_margin="20dp"/>
```

```
    </LinearLayout>
</androidx.cardview.widget.CardView>

<!-- This is extra space given to maintain a
        gap between two consecutive CardViews-->
<Space
        android:layout_width="match_parent"
        android:layout_height="10dp"/>
```

```
</LinearLayout>
```

LAYOUT-TWO.XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:id="@+id/linearlayout"
    >
    <!--parent CardView-->
    <androidx.cardview.widget.CardView
```

```
android:id="@+id/cardview_two"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="4dp"
android:layout_marginEnd="16dp"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintTop_toTopOf="parent"
android:elevation="5dp">
```

```
<!--LinearLayout inside the CardView-->
```

```
<!--This layout is accessed to create
      toasts when this item is clicked-->
```

```
<LinearLayout
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">
```

```
<!--This layout consists of an ImageView
      and two TextViews-->
```

```
<ImageView
```

```
    android:id="@+id/image"
    android:layout_width="80dp"
    android:layout_height="80dp"/>
```

```
<!-- The purpose of this LinearLayout is
      to align the two TextViews Vertically-->
```

```
<LinearLayout
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical">
```

```
<!--These are the 2 TextViews in this layout-->
```

```
<TextView
```

```
    android:id="@+id/text_one"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_marginTop="10dp"
```

```
    android:layout_marginStart="20dp"
```

```
    android:textColor="#000000"
```

```
    android:textSize="20dp"/>
```

```
<TextView
```

```
    android:id="@+id/text_two"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:textColor="#000000"
```

```
    android:textSize="15dp"
```

```
    android:layout_marginTop="10dp"
```

```
    android:layout_marginStart="20dp"/>
```

```
</LinearLayout>
```

```
</LinearLayout>
```

```
</androidx.cardview.widget.CardView>

<!-- This is extra space given to
      maintain a gap between two CardViews-->

<Space
      android:layout_width="match_parent"
      android:layout_height="10dp"/>
```

```
</LinearLayout>
```

Create an Item class

```
package com.example.android.multilayoutrecyclerview;
```

```
// ItemClass
```

```
public class ItemClass {
```

```
    // Integers assigned to each layout
    // these are declared static so that they can
    // be accessed from the class name itself
    // And final so that they are not modified later
    public static final int LayoutOne = 0;
    public static final int LayoutTwo = 1;

    // This variable ViewType specifies
    // which out of the two layouts
    // is expected in the given item
```

```
private int viewType;

// String variable to hold the TextView
// of the first item.
private String text;

// public constructor for the first layout
public ItemClass(int viewType, String text)
{
    this.text = text;
    this.viewType = viewType;
}

// getter and setter methods for the text variable

public String getText() { return text; }

public void setText(String text) { this.text = text; }

// Variables for the item of second layout
private int icon;
private String text_one, text_two;

// public constructor for the second layout
public ItemClass(int viewType, int icon, String text_one,
                String text_two)
```

```
{  
    this.icon = icon;  
    this.text_one = text_one;  
    this.text_two = text_two;  
    this.viewType = viewType;  
}  
  
// getter and setter methods for  
// the variables of the second layout  
  
public int geticon() { return icon; }  
  
public void seticon(int icon) { this.icon = icon; }  
  
public String getText_one() { return text_one; }  
  
public void setText_one(String text_one)  
{  
    this.text_one = text_one;  
}  
  
public String getText_two() { return text_two; }  
  
public void setText_two(String text_two)  
{  
    this.text_two = text_two;
```

```
    }

    public int getViewType() { return viewType; }

    public void setViewType(int viewType)
    {
        this.viewType = viewType;
    }
}
```

Create the Adapter class

```
package com.example.android.multilayoutrecyclerview;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

import static com.example.android.multilayoutrecyclerview.ItemClass.LayoutOne;
```

```
import static com.example.android.multilayoutrecyclerview.ItemClass.LayoutTwo;
```

```
public class AdapterClass extends RecyclerView.Adapter {
```

```
    private List<ItemClass> itemClassList;
```

```
    // public constructor for this class
```

```
    public AdapterClass(List<ItemClass> itemClassList)
```

```
    {
```

```
        this.itemClassList = itemClassList;
```

```
    }
```

```
    // Override the getItemViewType method.
```

```
    // This method uses a switch statement
```

```
    // to assign the layout to each item
```

```
    // depending on the viewType passed
```

```
    @Override
```

```
    public int getItemViewType(int position)
```

```
    {
```

```
        switch (itemClassList.get(position).getViewType()) {
```

```
            case 0:
```

```
                return LayoutOne;
```

```
            case 1:
```

```
                return LayoutTwo;
```

```
            default:
```

```
        return -1;

    }

}

// Create classes for each layout ViewHolder.

class LayoutOneViewHolder

    extends RecyclerView.ViewHolder {

        private TextView textview;

        private LinearLayout linearLayout;

        public LayoutOneViewHolder(@NonNull View itemView)

        {

            super(itemView);

            // Find the Views

            textview = itemView.findViewById(R.id.text);

            linearLayout

                = itemView.findViewById(R.id.linearlayout);

        }

        // method to set the views that will

        // be used further in onBindViewHolder method.

        private void setView(String text)

        {
```

```
        textview.setText(text);  
    }  
}
```

// similarly a class for the second layout is also
// created.

```
class LayoutTwoViewHolder  
    extends RecyclerView.ViewHolder {  
  
    private ImageView icon;  
    private TextView text_one, text_two;  
    private LinearLayout linearLayout;  
  
    public LayoutTwoViewHolder(@NonNull View itemView)  
    {  
        super(itemView);  
        icon = itemView.findViewById(R.id.image);  
        text_one = itemView.findViewById(R.id.text_one);  
        text_two = itemView.findViewById(R.id.text_two);  
        linearLayout  
            = itemView.findViewById(R.id.linearlayout);  
    }  
  
    private void setViews(int image, String textOne,
```

String textTwo)

```
{  
    icon.setImageResource(image);  
    text_one.setText(textOne);  
    text_two.setText(textTwo);  
}  
}
```

```
// In the onCreateViewHolder, inflate the  
// xml layout as per the viewType.  
// This method returns either of the  
// ViewHolder classes defined above,  
// depending upon the layout passed as a parameter.
```

```
@NonNull  
@Override  
public RecyclerView.ViewHolder  
onCreateViewHolder(@NonNull ViewGroup parent,  
                    int viewType)  
{  
    switch (viewType) {  
        case LayoutOne:  
            View layoutOne  
                = LayoutInflater.from(parent.getContext())  
                    .inflate(R.layout.layout_one, parent,  
                            false);
```

```
        return new LayoutOneViewHolder(layoutOne);

    case LayoutTwo:

        View layoutTwo

            = LayoutInflater.from(parent.getContext())

                .inflate(R.layout.layout_two, parent,

                    false);

        return new LayoutTwoViewHolder(layoutTwo);

    default:

        return null;

    }

}
```

```
// In onBindViewHolder, set the Views for each element
// of the layout using the methods defined in the
// respective ViewHolder classes.
```

```
@Override
```

```
public void onBindViewHolder(
```

```
    @NonNull RecyclerView.ViewHolder holder,
```

```
    int position)
```

```
{
```

```
    switch (itemClassList.get(position).getViewType()) {
```

```
        case LayoutOne:
```

```
            String text
```

```
        = itemClassList.get(position).getText();

        ((LayoutOneViewHolder)holder).setView(text);

// The following code pops a toast message
// when the item layout is clicked.
// This message indicates the corresponding
// layout.

        ((LayoutOneViewHolder)holder)

            .linearLayout.setOnClickListener(

                new View.OnClickListener() {

                    @Override

                    public void onClick(View view)

                    {

                        Toast

                            .makeText(

                                view.getContext(),

                                "Hello from Layout One!",

                                Toast.LENGTH_SHORT)

                            .show();

                    }

                });

        break;

case LayoutTwo:
```

```
        int image
            = itemClassList.get(position).geticon();

        String text_one
            = itemClassList.get(position).getText_one();

        String text_two
            = itemClassList.get(position).getText_two();

        ((LayoutTwoViewHolder)holder)
            .setViews(image, text_one, text_two);

        ((LayoutTwoViewHolder)holder)
            .linearLayout.setOnClickListener(
                new View.OnClickListener() {
                    @Override
                    public void onClick(View view)
                    {

                        Toast
                            .makeText(
                                view.getContext(),
                                "Hello from Layout Two!",
                                Toast.LENGTH_SHORT)
                            .show();
                    }
                });

        break;

    default:

        return;
```

```
        }  
    }  
  
    // This method returns the count of items present in the  
    // RecyclerView at any given time.  
  
    @Override  
    public int getItemCount()  
    {  
        return itemClassList.size();  
    }  
}
```

MainActivity.java

```
package com.example.android.multilayoutrecyclerview;  
  
import androidx.appcompat.app.AppCompatActivity;  
import androidx.recyclerview.widget.LinearLayoutManager;  
import androidx.recyclerview.widget.RecyclerView;  
  
import android.os.Bundle;  
import android.widget.Adapter;  
  
import java.util.ArrayList;  
import java.util.List;  
  
public class MainActivity extends AppCompatActivity {
```

```
@Override
protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // From the MainActivity, find the RecyclerView.
    RecyclerView recyclerView
        = findViewById(R.id.recyclerView);

    // Create and set the layout manager
    // For the RecyclerView.
    LinearLayoutManager layoutManager
        = new LinearLayoutManager(this);
    recyclerView.setLayoutManager(layoutManager);

    List<ItemClass> itemClasses = new ArrayList<>();

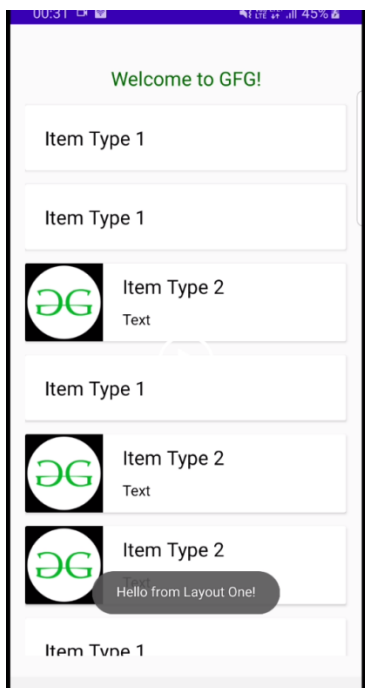
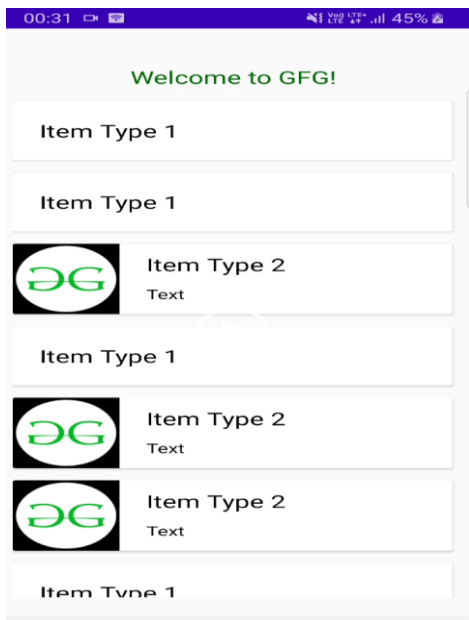
    // pass the arguments
    itemClasses.add(new ItemClass(ItemClass.LayoutOne,
                                   "Item Type 1"));
    itemClasses.add(new ItemClass(ItemClass.LayoutOne,
                                   "Item Type 1"));
    itemClasses.add(new ItemClass(
        ItemClass.LayoutTwo, R.drawable.icon,
```

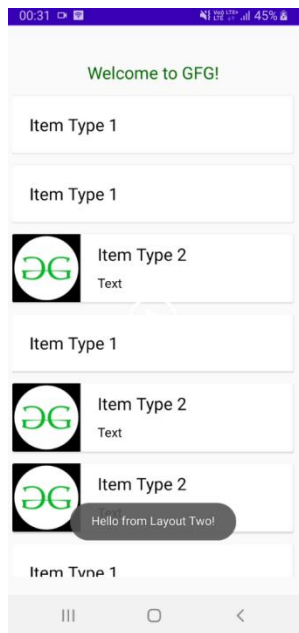
```
        "Item Type 2", "Text"));
itemClasses.add(new ItemClass(ItemClass.LayoutOne,
                                "Item Type 1"));
itemClasses.add(new ItemClass(
    ItemClass.LayoutTwo, R.drawable.icon,
    "Item Type 2", "Text"));
itemClasses.add(new ItemClass(
    ItemClass.LayoutTwo, R.drawable.icon,
    "Item Type 2", "Text"));
itemClasses.add(new ItemClass(ItemClass.LayoutOne,
                                "Item Type 1"));
itemClasses.add(new ItemClass(
    ItemClass.LayoutTwo, R.drawable.icon,
    "Item Type 2", "Text"));
AdapterClass adapterClass
    = new AdapterClass(itemClasses);

AdapterClass adapter
    = new AdapterClass(itemClasses);

// set the adapter
recyclerView.setAdapter(adapter);
}
}
```

OUTPUT:-





PRACTICAL-17

AIM:- Publish an Android Application on Play Store

Step 1: Create a Google Developer account

This is something you can do at the beginning of the app development process. Without registering a Google Developer Account, you can't publish your app on Google Play.

You can use any of your current Google accounts or create another one to sign up for a Google Developer Account. It doesn't matter whether it's a private or corporate account. You may easily transfer your app to another one in the future.

The creation process includes signing the Google Play Developer distribution agreement, adding some personal information, and paying a one-time registration fee of \$25. There is nothing complicated. Just follow the instructions.

The screenshot shows the Google Play Developer account creation interface. At the top, a progress bar has four steps: 'Sign-in with your Google account' (completed), 'Accept Developer Agreement' (active), 'Pay Registration Fee', and 'Complete your Account details'. Below the progress bar, the text 'YOU ARE SIGNED IN AS...' is followed by a user profile icon and a blurred name. To the right, a message states: 'This is the Google account that will be associated with your Developer Console. If you would like to use a different account, you can choose from the following options below. If you are an organization, consider registering a new Google account rather than using a personal account.' Below this message are two links: 'Sign in with a different account' and 'Create a new Google account'. A horizontal line separates this section from the next. The next section is titled 'BEFORE YOU CONTINUE...' and contains three columns of information. The first column has a document icon and text: 'Read and agree to the Google Play Developer distribution agreement.' Below this is a checkbox labeled 'I agree and I am willing to associate my account registration with the Google Play Developer distribution agreement.' The second column has a globe with a dollar sign icon and text: 'Review the distribution countries where you can distribute and sell applications. If you are planning to sell apps or in-app products, check if you can have a merchant account in your country.' The third column has a credit card icon and text: '\$25 Make sure you have your credit card handy to pay the \$25 registration fee in the next step.' At the bottom left of this section is a blue button labeled 'Continue to payment'.

Sign-in with your Google account Accept Developer Agreement Pay Registration Fee Complete your Account details

YOU ARE SIGNED IN AS...

 This is the Google account that will be associated with your Developer Console.
If you would like to use a different account, you can choose from the following options below. If you are an organization, consider registering a new Google account rather than using a personal account.
[Sign in with a different account](#) [Create a new Google account](#)

BEFORE YOU CONTINUE...



Read and agree to the [Google Play Developer distribution agreement](#).

☐ I agree and I am willing to associate my account registration with the Google Play Developer distribution agreement.



Review the distribution countries where you can distribute and sell applications.

If you are planning to sell apps or in-app products, check if you can have a merchant account in your country.

 \$25
Make sure you have your credit card handy to pay the \$25 registration fee in the next step.

[Continue to payment](#)

Google Play Developer distribution agreement

Usually, it takes no more than two days to get approval from Google. Don't worry if you forget to add some information. You can edit your account later.

Step 2: Add a Merchant Account

If you plan to sell paid apps or in-app purchases, you have to create a Google Merchant Account. There you can manage app sales and your monthly payouts, as well as analyze sales reports.

Once you finish creating the Merchant profile, the developer account gets automatically linked to it.

Step 3: Prepare the Documents

Paperwork always requires much effort, especially when it comes to any kind of legal documents. Based on our experience, we highly recommend starting to prepare the End User License Agreement (EULA) and Privacy Policy in advance.

You can take the documents from similar apps as references and create your own based on them, or ask a lawyer to make everything from scratch.

EULA is an agreement between you as an owner and a user of your product. In brief, it contains:

- What the users can do with the app, and what they aren't allowed to do
- Licensing fees
- Intellectual property information, etc.

Terms of Use or Terms and Conditions explain what services you offer the users and how you expect them to behave in return. Though Google doesn't demand Terms of Use, it's better to publish them. You can create one document, adding there Privacy Policy and Terms of Use chapters.

Pay special attention to include in the Privacy Policy the following information:

- A complete list of personal data that is collected, processed and used through the app
- Technical information that is collected about the device and the installed OS
- Functional features of the app, its paid and free functionality
- Place of registration of the company and/or location of the copyright holder of the application
- The chosen legal system and legislation that will be applied in resolving disputes and regulating legal relations
- The terms of subscription
- Citizenship (residence) of the overwhelming majority of application users
- Age criteria, the presence of specific content

Step 4: Study Google Developer Policies

We guess you already made up your product concept. Now it's time to make sure that every feature you will implement in the app is aligned with the Google Developer Policies. These documents explain how mobile apps need to be developed, updated, and promoted to support the store's high-quality standards.

If Google decides that your product violates some policy chapters, it may be rejected, blocked, or even deleted from Google Play. Besides, numerous and repetitive violations may lead to the developer account termination.

Read also: Why Google and Apple May Remove Your App and How to Deal With That

So study all the available information carefully about:

- Restricted content definition
- App store listing and promotion
- Impersonation and intellectual property
- Rules for monetization and ads
- Privacy, security and deception regulation
- Spam and minimum functionality

Google is constantly working on its policies, and it's important to monitor the changes and stay up to date even after your Android app is released.

Step 5: Technical Requirements

You went through the development process, endless testing, and bug fixing, and finally, the “X-day” comes. Before moving on to the upload process, you need to check the following things:

- Unique Bundle ID

The package name should be suitable over the life of your application. You cannot change it after the distribution. You can set the package name in the application's manifest file.

- Signed App Release With a Signing Certificate

Every application should be digitally signed with a developer's certificate. The certificate is used to identify the author of an app and can't be generated again.

- The App Size

Google set the limit size of the uploaded file: 100MB for Android 2.3 and higher (API level 9-10, 14 and higher) and 50MB for lower Android versions.

If your app exceeds this limit, you can always switch to APK Expansion Files.

- The File Format

Two possible release formats are accepted by Google: app bundle and .apk. However, .aab is the preferred one. To use this format, you need to enroll in app signing by Google Play.

You may learn more about app file technical requirements in the Developer Documents, Prepare for the release guide.

Step 6: Creating the App on the Google Console

Now you have the file that is ready for uploading. It's time to get to the fun part. Let's create a new app in your Developer Account:

- Reach to *All applications* tab in the menu
- Now select *Create Application*
- Choose the app's default language from the drop-down menu

- Add a brief app description (you can change it later)
- Tap on *Create*

After this, you will be taken to the store entry page, where we will add the complete data about the app.

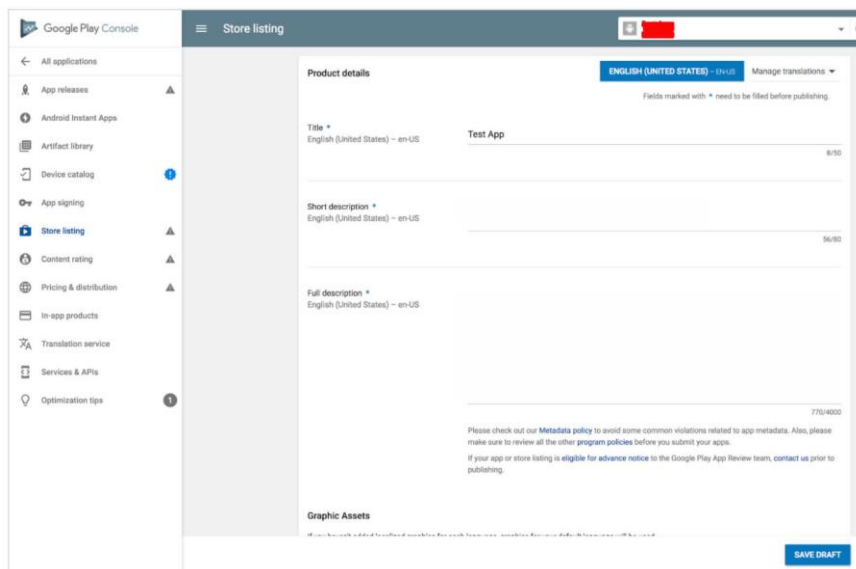
Step 7: Store Listing

First, let's prepare the Store listing. It contains the most important information useful for app store optimization (ASO) and gives the users more details about your app before downloading. The mandatory sections are marked with *.

You may need some designer and copywriter efforts, so it's better to start preparing the following materials in advance.

- Product description

It contains a title of your app (up to 50 symbols), a brief description (up to 80 symbols), and a full description (up to 4000 symbols). Control yourself and do not overdo the keywords.



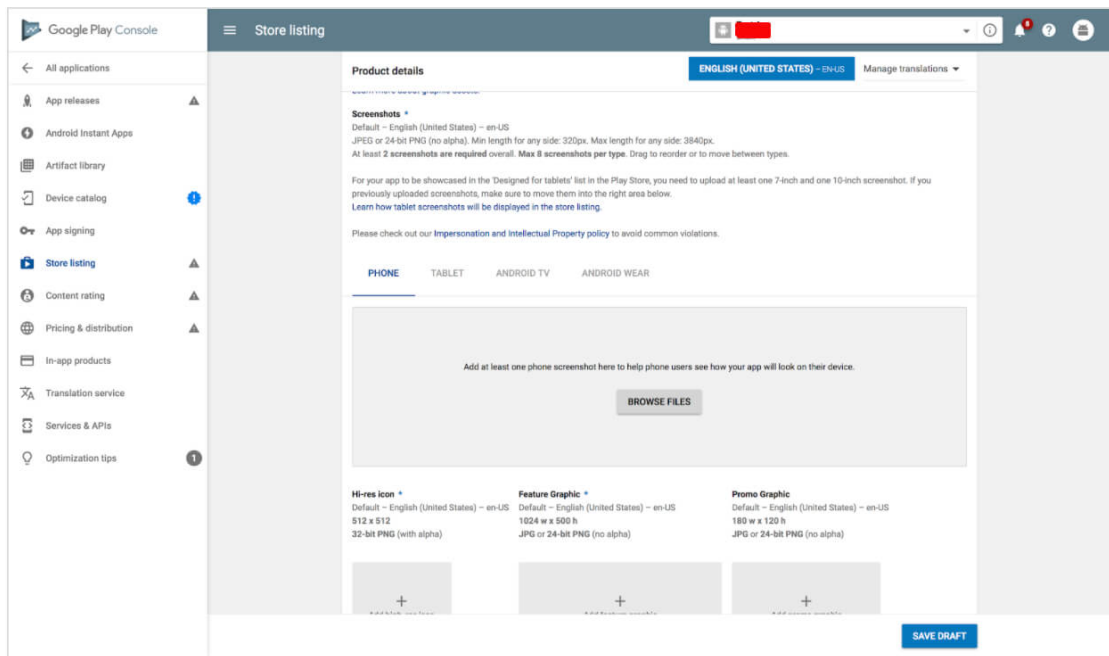
Store listing

- Screenshots

You may add from 2 to 8 screenshots. Choose the ones that show the app functionality and value the most.

The requirements are the following:

- JPEG or 24-bit PNG (no alpha)
- from 320px to 3840 px
- the ratio of the long side to the short side should not be more than 2:1



Store listing - Product details

- Icon

The requirements are the following:

- 512px by 512px
- 32-bit PNG (with alpha)
- Maximum file size: 1024KB

- Feature graphic

It is an optional marketing tool displayed in various places on Google Play, for example, on the homepage.

The requirements are the following:

- JPEG or 24-bit PNG (no alpha)
- 1024px x 500px

- Promo video

If you have any promo video, you may add a link to your YouTube channel. This video will be shown before the screenshots on the app's page.

- Tags

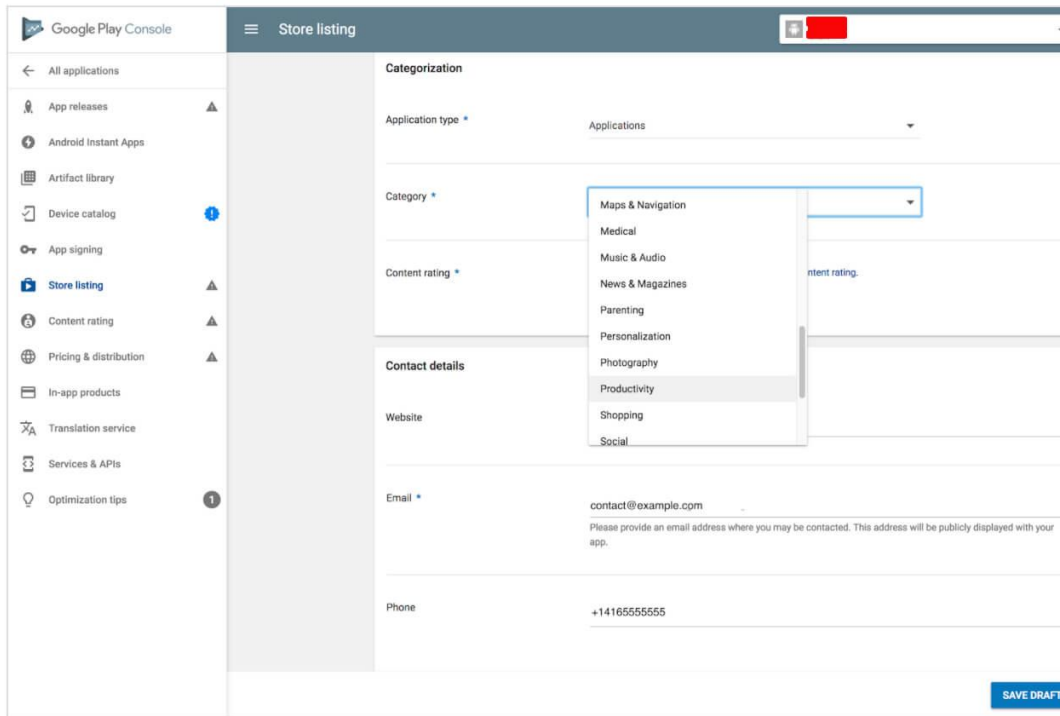
You may choose from the list the most relevant to your app keywords for better ASO. There is no possibility to add any custom tags.

- Localization

If your app supports several languages, mention all of them and add translations of your app's information. It's highly recommended to include localized screenshots and images.

- Application type and categorization

First, through the drop-down menu, select the application type: game or app. Then pick the category that your app fits into. You can also add a section to rate your content after uploading APK to Google Play store.

The image is a screenshot of the Google Play Console interface, specifically the 'Store listing' page for an application. On the left, there is a sidebar menu with various options like 'All applications', 'App releases', 'Android Instant Apps', 'Artifact library', 'Device catalog', 'App signing', 'Store listing' (which is highlighted), 'Content rating', 'Pricing & distribution', 'In-app products', 'Translation service', 'Services & APIs', and 'Optimization tips'. The main content area is titled 'Store listing' and contains several sections. The 'Categorization' section has a dropdown for 'Application type' set to 'Applications' and another dropdown for 'Category' which is currently open, showing a list of categories including 'Maps & Navigation', 'Medical', 'Music & Audio', 'News & Magazines', 'Parenting', 'Personalization', 'Photography', 'Productivity', 'Shopping', and 'Social'. Below this is the 'Content rating' section. The 'Contact details' section includes fields for 'Website', 'Email' (with the value 'contact@example.com'), and 'Phone' (with the value '+14165555555'). A 'SAVE DRAFT' button is located at the bottom right of the form.

App categories on Google Play

- Contact details

Here you should provide the support service contacts. By filling the website URL, email, and phone, you make it easier for the users to contact you if necessary.

- Privacy Policy

Google requires you to add a link to the Privacy Policy that we discussed above.

While editing the Store Listing, you can take a break at any moment, click *Save Draft*, and complete this stage later.

Step 8: Content Rating

In order not to be marked as an Unrated App (that may lead to app removal), pass a rating questionnaire. You can easily find this section on the left-side menu.

The information provided in the questionnaire must be accurate. Any misrepresentation of your app's content might lead to suspension or removal of the Google Play account.

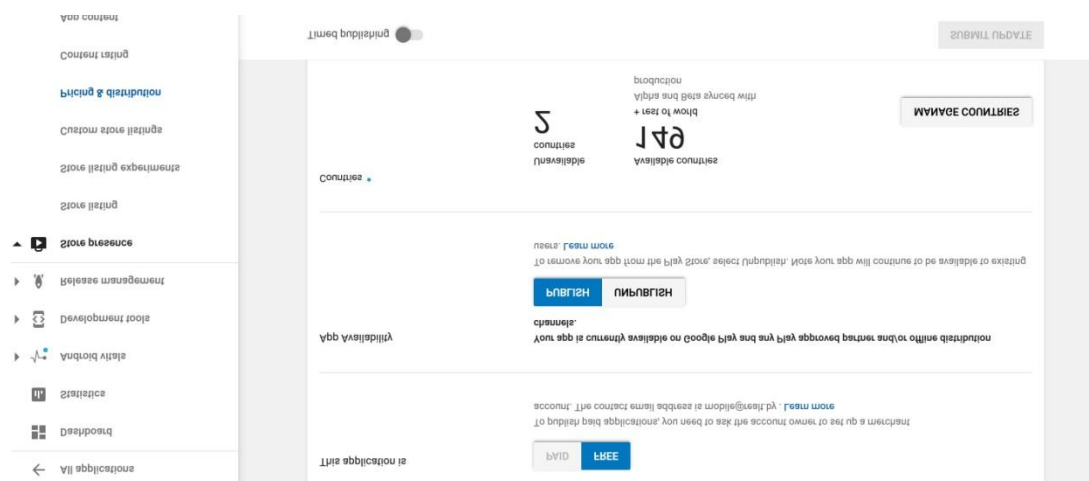
- Click on *Save Questionnaire* once you complete the survey
- Click on *Calculate Rating*
- In the end, click on *Apply Rating* to confirm the rating and move forward with the pricing & distribution plan

Step 9: Pricing the Application

In the *Pricing and distribution* section, you need to fill the following information:

- Whether your app is free or paid
- Where the app will be available (just choose the countries from the list)
- Whether your app will be available only on the specific devices
- Whether the app has sensitive content and is not suitable for children under the age of 13
- Whether your app contains ads

Remember that you can change your paid app to a free one later, but you cannot do the vice versa. If you decide later that you want to distribute the app for money, you'll have to create another app.



Store presence

Step 10: Upload APK and Send for Review

Finally, you are ready to upload your Android app file. That's the most exciting moment ever.

Let's go to the *App Releases* section on the left panel. Here you will find three options for publishing the app: Production, Beta and Alpha tracks.

We highly recommend starting with Alpha or Beta versions. In this case, after passing the review process, your app will not be available to everyone on the Google Play store.

The Alpha version assumes closed testing and is available only to those who you invite as testers. The Beta version means that anyone can join your testing program and send feedback to you.

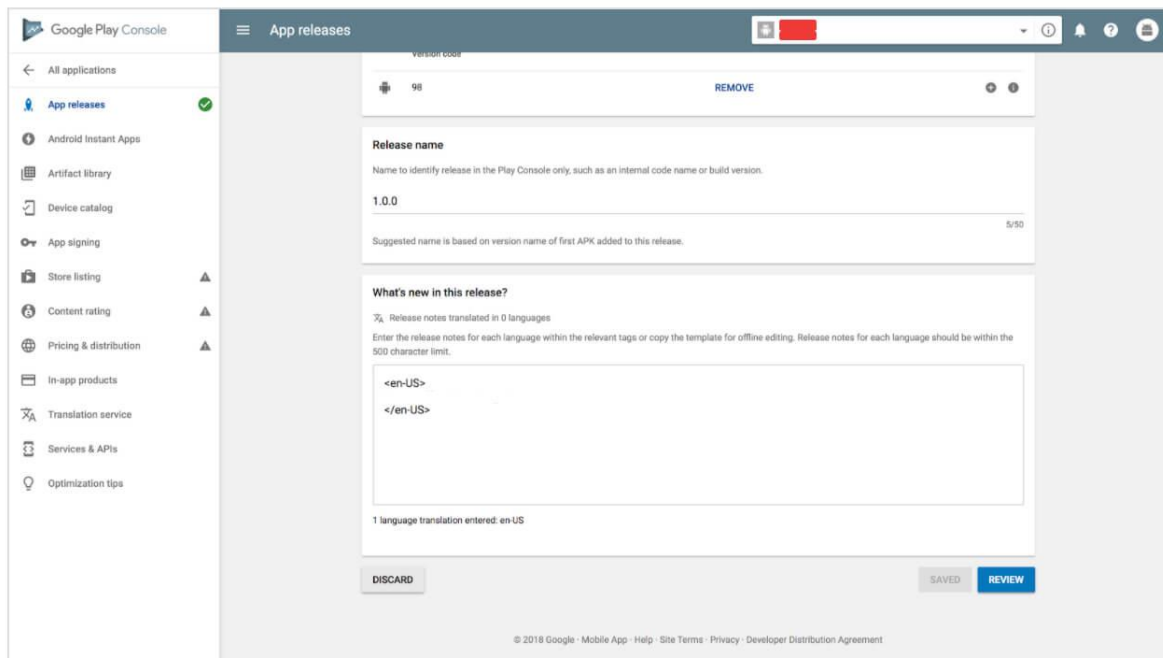
Pre-release testing allows you to gather people's opinions, test your app in a broader audience, and fix issues before making the app public.

Note that if you decide later to change the Alpha or Beta version to Production type, it will take time to go through another review round.

Once you choose the type of release, follow the steps:

- Choose *Manage (Production/Beta/Alpha)*
- Click on *Edit Release*
- Upload an APK or app bundle

The release name will be added automatically. For the first time, you may delete the text from the *What's new in this release* field.



App releases

- Click on *Review* to confirm the changes and send your app to the review by pressing *Start rollout to production*.

Don't worry that you may forget to add some information. All the way, Google will show you the instructions and tips. Actually, you won't manage to send the app to the review if something important is missed.

Remember that with the very first version, there is no opportunity to set manual publishing. The app will be released right after it passes the review. Usually, it takes up to 2 days. Google says the review process could take up to 7 days or even longer.

Once the app is reviewed, you'll receive a notification on Google Console Dashboard.

